Pedro Henrique Gomes

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

São Paulo
2026

Pedro Henrique Gomes

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Prof. Afonso Brandão

São Paulo
2026

Resumo

A crescente disseminação de dispositivos de exibição com elevadas taxas de atualização (60 Hz a 144 Hz) evidenciou a limitação temporal de conteúdos audiovisuais capturados nas frequências tradicionais de 24 FPS a 30 FPS. Para mitigar artefatos visuais e aumentar a fluidez contínua, a Interpolação de Quadros de Vídeo (VFI – *Video Frame Interpolation*) busca sintetizar quadros intermediários. Embora abordagens baseadas em fluxo óptico denso, como o modelo RAFT, alcancem elevada precisão, a alta complexidade computacional e a dependência de *frameworks* de alto nível (PyTorch) limitam sua aplicação sob restrições severas de hardware. Como alternativa eficiente, as Convoluções Adaptativas Separáveis tratam a síntese como uma filtragem local adaptativa, decompondo *kernels* bidimensionais $N \times N$ em dois filtros unidimensionais $1D$, reduzindo a complexidade aritmética de $O(N^2)$ para $O(2N)$ por pixel. Este trabalho apresenta uma análise experimental do modelo SepConv através da implementação de um motor de inferência nativo em C++ e CUDA, desacoplado de ecossistemas de treinamento. O *pipeline* proposto integra a biblioteca OpenCV para entrada e saída de dados, a API C++ do ONNX Runtime para a extração dos vetores de movimento e *kernels* customizados em CUDA para a execução paralela das convoluções mantidas diretamente na memória de vídeo (VRAM). A validação experimental prevê a avaliação do compromisso entre o consumo de memória VRAM (aferido via NVML e focado em GPUs de consumo de 8 GB), a latência de execução em milissegundos e a fidelidade visual (mensurada por PSNR e SSIM sobre o *dataset* Vimeo-90K). Os resultados buscam comprovar a viabilidade técnica do controle granular de memória para viabilizar a interpolação de vídeos em alta definição em hardwares comerciais restritos.

Palavras-chave: Interpolação de Quadros de Vídeo. Convoluções Adaptativas Separáveis. C++. CUDA. Otimização de VRAM. SepConv.

Abstract

The growing dissemination of display devices with high refresh rates (60 Hz to 144 Hz) has highlighted the temporal limitations of audiovisual content captured at traditional frame rates of 24 FPS to 30 FPS. To mitigate visual artifacts and enhance temporal smoothness, Video Frame Interpolation (VFI) aims to artificially synthesize intermediate frames. Although dense optical flow-based approaches, such as the RAFT model, achieve high accuracy, their excessive computational complexity and dependence on high-level frameworks (PyTorch) constrain their deployment under severe hardware restrictions. As an efficient alternative, Adaptive Separable Convolutions model frame synthesis as adaptive local filtering, factorizing $N \times N$ two-dimensional kernels into two $1D$ line-separable filters, thereby reducing arithmetic complexity from $O(N^2)$ to $O(2N)$ per pixel. This work presents an experimental analysis of the SepConv model through the implementation of a native C++ and CUDA inference engine, completely decoupled from training ecosystems. The proposed pipeline integrates the OpenCV library for data input/output, the ONNX Runtime C++ API for extracting motion vectors, and custom CUDA kernels for the parallel execution of convolutions maintained directly within video memory (VRAM). Experimental validation evaluates the trade-off between VRAM memory consumption (measured via NVML, focusing on consumer-grade 8 GB GPUs), execution latency in milliseconds, and visual fidelity (quantified using PSNR, SSIM, and VMAF on the Vimeo-90K dataset). The results aim to demonstrate the technical viability of granular memory management to enable high-definition video frame interpolation on resource-constrained commercial hardware.

Keywords: Video Frame Interpolation. Adaptive Separable Convolutions. C++. CUDA. VRAM Optimization. SepConv.

Lista de Abreviaturas e Siglas

A lista a seguir reúne, em ordem alfabética, as abreviaturas e siglas utilizadas neste trabalho, acompanhadas de seus respectivos significados.

Abreviatura/Sigla	**Significado**
ABNT	Associação Brasileira de Normas Técnicas
ALU	*Arithmetic Logic Unit* (Unidade Lógica e Aritmética)
API	*Application Programming Interface* (Interface de Programação de Aplicações)
CLI	*Command-Line Interface* (Interface de Linha de Comando)
CPU	*Central Processing Unit* (Unidade Central de Processamento)
CUDA	*Compute Unified Device Architecture*
EP	*Execution Provider* (Provedor de Execução do ONNX Runtime)
FFV1	*FFmpeg Video Codec 1* (codec de vídeo sem perdas)
FLOPs	*Floating Point Operations* (Operações de Ponto Flutuante)
FP16	Ponto flutuante de 16 bits (meia precisão)
FPS	*Frames Per Second* (Quadros por Segundo)
GPGPU	*General-Purpose Computing on Graphics Processing Units*
GPU	*Graphics Processing Unit* (Unidade de Processamento Gráfico)
I/O	*Input/Output* (Entrada e Saída)
MiB	*Mebibyte* (2^{20} bytes)
MSE	*Mean Squared Error* (Erro Quadrático Médio)
NVML	*NVIDIA Management Library*
ONNX	*Open Neural Network Exchange*
PCIe	*Peripheral Component Interconnect Express*
PSNR	*Peak Signal-to-Noise Ratio*
RAFT	*Recurrent All-Pairs Field Transforms*
RAM	*Random Access Memory*
SIMT	*Single Instruction, Multiple Threads*
SM	*Streaming Multiprocessor*
SSIM	*Structural Similarity Index*
TCC	Trabalho de Conclusão de Curso
TF32	*TensorFloat-32*
VFI	*Video Frame Interpolation* (Interpolação de Quadros de Vídeo)
VMAF	*Video Multimethod Assessment Fusion*
VRAM	*Video Random Access Memory*

Sumário

Sumário **5**

1 INTRODUÇÃO **7**
1.1 Contexto **8**
1.2 Formulação do problema **9**
1.3 Objetivos e hipótese **9**
1.3.1 Objetivo geral 9
1.3.2 Objetivos específicos 9
1.3.3 Hipótese e critérios de validação 10
1.4 Justificativa **10**

2 REVISÃO BIBLIOGRÁFICA **12**
2.1 Metodologia da pesquisa bibliográfica **12**
2.1.1 Período de abrangência 12
2.1.2 Critérios de inclusão 13
2.1.3 Critérios de exclusão 13
2.2 Fundamentos **14**
2.2.1 Definição de Imagem Digital 14
2.2.2 Filtragem Espacial e Convolução Adaptativa 15
2.2.3 Fluxo Óptico Explícito vs. Kernels de Movimento Implícito 16
2.2.4 Eficiência via Convoluções Separáveis 17
2.2.5 Implementação de Baixo Nível e I/O de Vídeo 18
2.2.6 Inferência Desacoplada via ONNX Runtime 19
2.2.7 Arquitetura de Decomposição e Garantia de Separabilidade 19
2.2.8 Arquitetura de GPU e o Gargalo de Memória (*Memory Wall*) 20
2.2.9 Modelo de Execução Paralela SIMT 20
2.2.10 Processamento de Baixo Nível e a Plataforma CUDA 21
2.2.11 Métricas de Avaliação Objetiva de Qualidade 22
2.2.12 Mensuração de Desempenho Computacional 23
2.3 Trabalhos relacionados **24**
2.3.1 Trabalho 1: Revisiting Adaptive Convolutions for Video Frame Interpolation 24
2.3.2 Trabalho 2: Motion-Aware Video Frame Interpolation 25
2.3.3 Trabalho 3: RAFT: Recurrent All-Pairs Field Transforms for Optical Flow . 25

3 DESENVOLVIMENTO **27**
3.1 Solução Proposta e Arquitetura **27**

3.2 Ambiente Experimental . . . 28
3.3 Exportação do Modelo e Grafo ONNX . . . 28
3.3.1 Correção do grafo para resoluções arbitrárias . . . 30
3.4 Implementação do Motor de Inferência . . . 30
3.4.1 Integração com o ONNX Runtime . . . 31
3.4.2 *Kernel* de convolução separável . . . 31
3.4.3 Instrumentação . . . 32
3.4.4 Equivalência com a implementação original . . . 32
3.5 Validação Experimental e Inspeção de Quadros . . . 33
3.5.1 Sequenciamento de Quadros Interpolados . . . 33
3.5.2 Inspeção de Pixels e Limites de Filtragem . . . 35
3.6 Protocolo Experimental . . . 35
3.6.1 Preparação dos vídeos de teste . . . 35
3.6.2 Procedimento de medição . . . 36
3.6.3 Métricas . . . 36
3.7 Resultados Preliminares . . . 37
3.7.1 Latência . . . 37
3.7.2 Consumo de VRAM . . . 39
3.7.3 Fidelidade visual . . . 40
3.7.4 Discussão e próximos passos . . . 44
3.8 Cronograma de trabalho . . . 46

REFERÊNCIAS . . . 47

1 Introdução

Historicamente, os sistemas de captura e exibição de imagens em movimento foram limitados pela capacidade mecânica e elétrica dos dispositivos da época. Durante a consolidação do cinema sonoro no início do século XX, a taxa de aproximadamente 24 quadros por segundo (FPS) tornou-se um padrão industrial por representar um equilíbrio entre fluidez perceptiva e custo de armazenamento e reprodução do filme físico (BORDWELL; THOMPSON; SMITH, 2016). Taxas nessa faixa passaram a ser suficientes para produzir a percepção de movimento contínuo ao sistema visual humano, reduzindo a percepção de descontinuidade temporal durante a projeção (WATKINSON, 2009).

Com a evolução tecnológica dos dispositivos de exibição, monitores e televisores modernos passaram a operar em frequências significativamente superiores, variando entre 60 Hz e 144 Hz. Entretanto, grande parte do conteúdo audiovisual disponível ainda permanece limitada às taxas tradicionais de captura, como 24 FPS ou 30 FPS, criando uma discrepância entre a capacidade de atualização do hardware atual e a quantidade de informação temporal presente no vídeo original. Essa diferença entre a taxa de amostragem temporal do conteúdo original e a frequência de atualização do dispositivo de exibição resulta no fenômeno conhecido como subamostragem temporal. De acordo com Watkinson (2009) e Szeliski (2022), a incapacidade de fornecer amostras intermediárias suficientes durante movimentos rápidos reduz a percepção de continuidade temporal pelo sistema visual humano, induzindo a artefatos perceptuais como tremulação de movimento (*motion judder*), borramento dinâmico e quebra na percepção de fluidez contínua.

Nesse contexto, a Interpolação de Quadros de Vídeo (VFI – *Video Frame Interpolation*) surge como uma técnica essencial para sintetizar quadros intermediários artificialmente, aumentando a fluidez temporal do conteúdo exibido. Tradicionalmente, a literatura divide as abordagens de VFI em métodos baseados em fluxo óptico, como o modelo RAFT (TEED; DENG, 2020), e métodos baseados em *kernels*. Embora modelos de aprendizado profundo tenham atingido patamares elevados de precisão, eles frequentemente demandam recursos computacionais extensivos e dependem de *frameworks* pesados, o que limita sua aplicação em cenários de hardware restrito ou escassez de memória RAM e VRAM.

Para mitigar a alta demanda de recursos computacionais — representada pelo elevado volume de operações de ponto flutuante (FLOPs), pelo esgotamento do armazenamento de tensores na memória de vídeo (VRAM) e pela alta latência de execução —, a literatura recente busca alternativas mais eficientes aos modelos tradicionais de fluxo óptico. Em cenários de hardware restrito, típicos de computadores pessoais e dispositivos de consumo doméstico dotados de GPUs comerciais limitadas a 8 GB de VRAM, a execução de ecossistemas densos baseados em *frameworks* de alto nível (como PyTorch) torna-se inviável em altas resoluções. Nesse contexto, Niklaus, Mai e Liu (2017) propuseram a utilização

de Convoluções Adaptativas Separáveis, técnica que decompõe *kernels* bidimensionais $N \times N$ em pares de filtros unidimensionais $1D$, reduzindo a complexidade computacional de $O(N^2)$ para $O(2N)$. A implementação dessa etapa de inferência em linguagem de baixo nível (C++/CUDA) possibilita um controle granular sobre o gerenciamento de memória gráfica, permitindo isolar a eficiência do algoritmo e testar sua viabilidade em computadores comuns.

Diante desse gargalo tecnológico entre a necessidade de fluidez em computadores pessoais e as limitações físicas de memória do hardware de consumo, formaliza-se a seguinte **hipótese científica**: postula-se que a separabilidade matemática dos *kernels*, quando associada a um *pipeline* de inferência nativo em C++ e CUDA (desacoplado de *frameworks* de treinamento), reduz o pico de consumo de VRAM e a latência de execução em comparação às implementações de alto nível em Python/PyTorch. A partir dessa premissa, o presente trabalho tem como objetivo avaliar a viabilidade técnica do processamento de vídeos em alta definição sob o limite de 8 GB de VRAM, investigando o equilíbrio entre a eficiência computacional no hardware do usuário final e a fidelidade visual dos quadros sintetizados.

1.1 Contexto

Historicamente, os padrões de captura e transmissão de vídeo foram consolidados sob limitações técnicas de largura de banda e processamento da época, estabelecendo taxas como 24 ou 30 quadros por segundo (FPS) como normas globais (GONZALEZ; WOODS, 2009). Contudo, a evolução recente dos dispositivos de exibição permitiu o surgimento de telas com altas taxas de atualização (60Hz a 144Hz), criando uma disparidade entre o conteúdo disponível e a capacidade do hardware. Essa lacuna resulta em uma percepção de movimento descontínua, pois a amostragem temporal reduzida do conteúdo original não preenche os intervalos de atualização das telas modernas, comprometendo a fluidez visual esperada pelo usuário.

Diferente de métodos simplificados de amostragem de quadros, a VFI busca sintetizar um novo quadro no instante $t+0.5$ por meio de uma combinação ponderada de informações espaciais e temporais das imagens adjacentes. Esse processo considera o contexto do movimento e a correlação entre pixels (NIKLAUS; MAI; LIU, 2017). Ao elevar a taxa de quadros por segundo, a técnica objetiva atenuar artefatos de movimento na percepção humana.

Entretanto, a implantação prática desses algoritmos enfrenta barreiras atreladas ao custo computacional espacial e aritmético. Modelos de estado da arte demandam alto armazenamento de tensores intermediários na memória de vídeo (VRAM), limitando frequentemente sua execução a hardwares de alto desempenho (BAO et al., 2019). A transposição de modelos matemáticos para pipelines de inferência otimizados surge como

um caminho científico relevante para avaliar os reais gargalos de desempenho em sistemas com memória de consumo limitada.

1.2 Formulação do problema

Apesar da elevada precisão atingida por modelos modernos de Interpolação de Quadros de Vídeo (VFI), a dependência de *frameworks* de alto nível (como PyTorch) e a alta demanda por memória de vídeo impedem sua execução fluida e eficiente em computadores pessoais munidos de GPUs comerciais. Diante dessa limitação física e arquitetural, formula-se a seguinte questão de pesquisa:

> *Em que medida a implementação de um motor de inferência nativo em C++ e CUDA para o modelo SepConv, fundamentado na separabilidade matemática de kernels $O(N)$, é capaz de reduzir a latência de processamento e o consumo de VRAM em GPUs de consumo de 8 GB, mantendo níveis adequados de fidelidade visual?*

1.3 Objetivos e hipótese

Esta seção delimita o escopo técnico da pesquisa, focando na avaliação de desempenho sob restrições físicas de VRAM em hardware de consumo.

1.3.1 Objetivo geral

Avaliar experimentalmente os efeitos da convolução separável no desempenho computacional da inferência do modelo SepConv em GPU, utilizando uma implementação nativa em C++ e CUDA, com ênfase na latência de execução e na mitigação do pico de consumo de memória VRAM em hardware de consumo.

1.3.2 Objetivos específicos

1. Desenvolver um módulo de Entrada e Saída (I/O) em C++ e OpenCV para extração e linearização de quadros de vídeo na memória principal (Host) e transferência contígua para a memória gráfica (Device);
2. Integrar a API C++ do *ONNX Runtime* (operando via *CUDA Execution Provider*) para carregar o modelo SepConv compilado e inferir nativamente na VRAM os vetores unidimensionais de movimento ($1D$);
3. Projetar e otimizar *kernels* customizados na arquitetura CUDA para aplicar a operação de convolução separável de ordem linear $O(N)$ mantida inteiramente na VRAM;

4. Desenvolver um ambiente de testes comparativo entre a implementação nativa proposta (C++/CUDA) e a *baseline* original do modelo SepConv em Python/PyTorch;

5. Quantificar a eficiência computacional por meio do monitoramento da latência por quadro (em milissegundos via `cudaEventRecord`), da taxa de quadros por segundo (FPS) e do pico de ocupação de VRAM (via biblioteca NVML) para diferentes resoluções de vídeo;

6. Avaliar a fidelidade visual e a consistência temporal dos quadros sintetizados por meio das métricas objetivas Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM) e Video Multimethod Assessment Fusion (VMAF) sobre o *dataset* Vimeo-90K.

1.3.3 Hipótese e critérios de validação

Postula-se como **hipótese científica** que a separabilidade matemática dos *kernels* de movimento ($O(N)$), quando associada a um *pipeline* de inferência nativo em C++ e CUDA desacoplado do *runtime* do Python, reduz significativamente o consumo de memória VRAM e a latência por quadro em comparação com a implementação de alto nível. O critério de validação do trabalho será atingido se o motor proposto for capaz de processar vídeos em alta definição sob o limite de 8 GB de VRAM, registrando ganho de vazão (FPS) e redução do consumo de memória sem causar degradação estatisticamente relevante nas métricas de fidelidade visual (PSNR, SSIM e VMAF).

1.4 Justificativa

A escolha pela investigação e implementação de algoritmos de interpolação baseados em Convoluções Adaptativas Separáveis justifica-se, primordialmente, pela necessidade de viabilizar a fluidez temporal em computadores pessoais com recursos de memória gráfica restritos. Enquanto abordagens de convolução espacial bidimensional completa demandam uma complexidade quadrática de N^2 operações por pixel para a síntese de quadros, a decomposição em *kernels* unidimensionais reduz esse custo para uma ordem linear de $2N$. Essa otimização matemática reduz o volume de operações e, fundamentalmente, o *footprint* (consumo espacial) de tensores mantidos na memória de vídeo (VRAM), viabilizando que hardwares comerciais processem vídeos em alta definição.

A delimitação experimental em GPUs de consumo dotadas de 8 GB de VRAM fundamenta-se na representatividade desse perfil de hardware entre usuários domésticos. Segundo dados da *Steam Hardware & Software Survey*, placas gráficas com 8 GB de memória representam aproximadamente 26% das configurações ativas no mercado consumidor. Embora modelos corporativos e GPUs de alto desempenho (com 16 GB a 80

GB de VRAM) tenham registrado avanço, a faixa de 8 GB constitui o principal gargalo físico para a execução de modelos densos de inteligência artificial fora de data centers.

Sob a perspectiva da Ciência da Computação e da Engenharia de Software, este trabalho justifica-se pela análise do gargalo imposto por *frameworks* de alto nível (como Python e PyTorch). Embora adequadas para fase de treinamento e prototipagem, essas linguagens introduzem *overheads* decorrentes de alocações dinâmicas imprevisíveis de memória, gerenciamento de *garbage collection* e interpretação em tempo de execução. Ao transpor a etapa de inferência do modelo SepConv para um ambiente nativo em C++, integrado à API do ONNX Runtime e a *kernels* customizados em CUDA, é possível exercer um controle determinístico sobre a alocação de ponteiros diretamente na VRAM.

Em suma, a justificativa do estudo reside na demonstração prática de que o gerenciamento granular de hardware, aliado à otimização algorítmica de baixo nível, permite superar limitações físicas de memória. Essa abordagem oferece um ponto de equilíbrio viável entre a qualidade visual da interpolação e a eficiência computacional em sistemas comerciais de consumo.

2 Revisão Bibliográfica

A revisão bibliográfica tem como finalidade apresentar o embasamento teórico do trabalho, demonstrando o conhecimento do autor sobre o tema e identificando o estado da arte. Este capítulo deve ser organizado por conceitos e temas, construindo uma linha de argumentação que sustente a proposta do trabalho.

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida por meio de buscas nas bases Google Scholar e IEEE Xplore, utilizando combinações de palavras-chave relacionadas ao tema da pesquisa. As consultas empregaram operadores lógicos para refinar os resultados, buscando trabalhos relacionados à interpolação de quadros de vídeo, convoluções separáveis, fluxo óptico e otimização de inferência em GPU.

- **Bases de dados consultadas**: Google Scholar e IEEE Xplore;
- **Palavras-chave utilizadas**:
  - ("video frame interpolation"OR "frame interpolation") AND ("convolutional neural network"OR CNN OR convolution) AND (video OR image sequence);
  - ("video frame interpolation") AND (convolution OR "adaptive convolution"OR "separable convolution");
  - ("video frame interpolation") AND ("optical flow"OR convolution OR transformer OR "deep learning");

2.1.1 Período de abrangência

Embora as buscas tenham retornado publicações de diferentes períodos, foi adotado como critério principal a seleção de trabalhos publicados a partir de 2015. Esse recorte temporal foi definido por contemplar o surgimento e a consolidação das principais técnicas modernas de aprendizado profundo aplicadas à interpolação de quadros de vídeo. Trabalhos anteriores foram mantidos apenas quando considerados referências clássicas ou fundamentais para a compreensão dos conceitos teóricos utilizados nesta pesquisa.

O processo de busca inicial retornou aproximadamente 5.000 resultados. Em uma primeira etapa de triagem (*screening*), foi realizada a leitura dos títulos dos trabalhos encontrados, resultando na seleção de aproximadamente 60 publicações potencialmente relevantes ao tema investigado. Após a aplicação do filtro temporal, cerca de 20 artigos permaneceram para análise.

Na etapa seguinte, foi realizada a leitura dos resumos dos trabalhos selecionados, reduzindo o conjunto para aproximadamente 12 publicações. Por fim, foi conduzida uma leitura mais aprofundada dos artigos com maior aderência ao escopo da pesquisa, dos quais cerca de cinco foram efetivamente utilizados na construção do referencial teórico e três foram selecionados para compor a seção de trabalhos relacionados.

2.1.2 Critérios de inclusão

Foram considerados elegíveis os trabalhos que atendessem a pelo menos um dos seguintes critérios:

- abordagem relacionada à interpolação de quadros de vídeo (*Video Frame Interpolation*);
- utilização de convoluções adaptativas ou convoluções separáveis;
- aplicação de técnicas de fluxo óptico para síntese temporal de quadros;
- estudos envolvendo inferência acelerada por GPU;
- pesquisas relacionadas à otimização computacional, consumo de memória ou redução de latência em aplicações de visão computacional;
- publicações com relevância científica comprovada por meio de conferências ou periódicos reconhecidos na área.

2.1.3 Critérios de exclusão

Foram excluídos da análise trabalhos que apresentassem uma ou mais das seguintes características:

- baixa aderência ao problema investigado;
- foco exclusivo na etapa de treinamento de modelos neurais;
- dependência excessiva de hardware específico sem aplicabilidade ao contexto da pesquisa;
- ausência de descrição técnica suficiente para reprodução ou compreensão da metodologia proposta;
- duplicidade entre bases consultadas;
- trabalhos que não abordassem aspectos relacionados à interpolação de quadros, fluxo óptico, convoluções separáveis ou otimização de inferência.

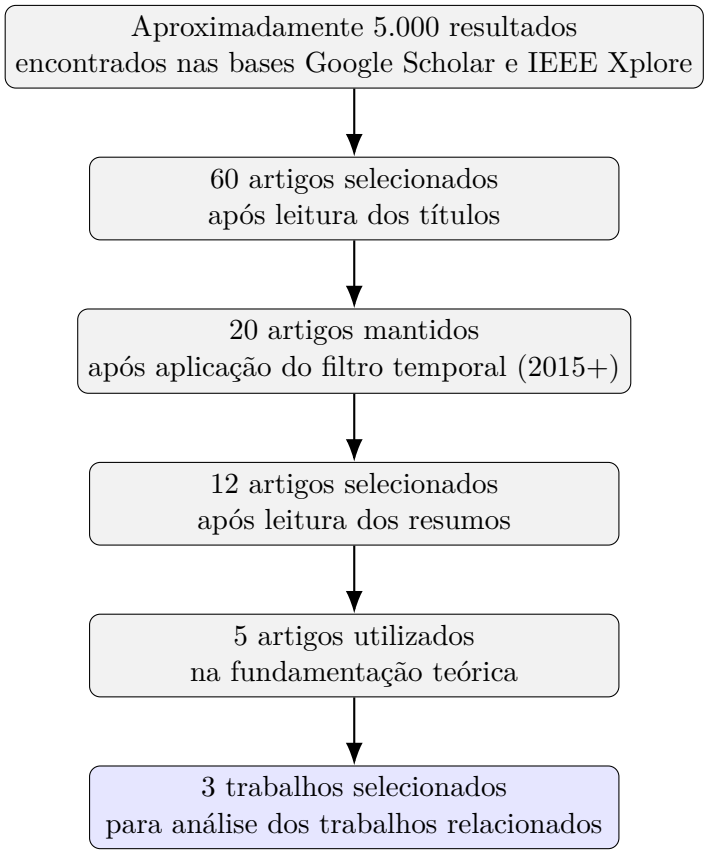


Figura 1 – Processo de seleção dos trabalhos utilizados na pesquisa bibliográfica.

Fonte: Elaborado pelo autor (2026)

A estratégia de seleção adotada permitiu reduzir progressivamente o conjunto inicial de publicações, mantendo apenas trabalhos com elevada aderência técnica ao objetivo da pesquisa e representativos do estado da arte em interpolação de quadros de vídeo e otimização de inferência em GPU.

2.2 Fundamentos

2.2.1 Definição de Imagem Digital

Uma imagem digital pode ser definida como uma função bidimensional $f(x, y)$, onde x e y representam coordenadas espaciais em um plano. De acordo com Gonzalez e Woods (2009), quando os valores de x, y e a amplitude de f assumem quantidades finitas e discretas, a imagem é caracterizada como digital. Na prática computacional, essa estrutura é representada por uma matriz composta por elementos discretos denominados *pixels* (*picture elements*), que constituem a menor unidade de informação visual de uma imagem.

No modelo de cores RGB (*Red, Green, Blue*), amplamente utilizado em sistemas computacionais, cada pixel é representado por três componentes correspondentes aos

canais vermelho, verde e azul. Em implementações convencionais de 8 bits por canal, cada componente assume valores no intervalo $[0, 255]$, permitindo a representação de aproximadamente 16,7 milhões de combinações de cores (SZELISKI, 2022).

A resolução de uma imagem corresponde à quantidade total de pixels que a compõem. Em aplicações de vídeo, o volume de dados cresce rapidamente com o aumento da resolução. Por exemplo, um único quadro em resolução Full HD (1920×1080) contém mais de dois milhões de pixels. Considerando três canais de cor por pixel, a manipulação simultânea de múltiplos quadros pode demandar dezenas de megabytes de memória, tornando o gerenciamento eficiente dos dados um aspecto relevante para algoritmos de processamento de vídeo e interpolação de quadros (SZELISKI, 2022).

A representação dessas matrizes em linguagem C++ impacta diretamente a performance computacional. Embora conceitualmente bidimensionais, as imagens são frequentemente organizadas na memória como vetores unidimensionais para maximizar o acesso contíguo aos dados. Nesse formato de linearização, a posição de um pixel (x, y) em uma imagem de largura W pode ser obtida pela expressão:

$$index = (y \times W) + x$$

Essa organização favorece a localidade espacial de memória e reduz a ocorrência de acessos não contíguos, característica particularmente importante em aplicações de processamento paralelo executadas em GPUs (SZELISKI, 2022; KIRK; HWU, 2023). Em algoritmos de interpolação de quadros, nos quais milhões de operações são realizadas para cada sequência de vídeo processada, a forma de armazenamento dos pixels pode influenciar diretamente a latência de execução e o consumo de memória.

2.2.2 Filtragem Espacial e Convolução Adaptativa

A convolução constitui uma das operações fundamentais do processamento digital de imagens e da visão computacional moderna. Em sua forma tradicional, um kernel bidimensional percorre toda a imagem aplicando o mesmo conjunto de coeficientes a cada região analisada. Segundo Gonzalez e Woods (2009), filtros convolucionais clássicos utilizam coeficientes fixos para executar tarefas como suavização, realce de detalhes, remoção de ruído e detecção de bordas.

Matematicamente, a convolução espacial pode ser interpretada como uma operação de combinação ponderada entre os pixels de uma vizinhança local. Em aplicações convencionais de processamento de imagens, a utilização de um kernel fixo é suficiente, uma vez que a transformação desejada é aplicada uniformemente sobre toda a cena. Entretanto, essa abordagem torna-se limitada em problemas de Interpolação de Quadros de Vídeo (VFI), nos quais diferentes regiões da imagem podem apresentar movimentos distintos entre quadros consecutivos (SZELISKI, 2022).

Em cenários de VFI, o objetivo não consiste apenas em filtrar uma imagem existente, mas em sintetizar novas informações visuais correspondentes a um instante temporal intermediário. Dessa forma, um único kernel fixo não é capaz de representar adequadamente a variedade de movimentos presentes na cena. Objetos distintos podem deslocar-se em velocidades e direções diferentes, exigindo estratégias adaptativas para estimar corretamente a posição dos pixels no quadro intermediário.

Para superar essa limitação, surgem as convoluções adaptativas, nas quais os coeficientes do kernel deixam de ser constantes e passam a ser estimados individualmente para cada pixel da imagem. Conforme proposto por Niklaus, Mai e Liu (2017), os coeficientes desses kernels funcionam como indicadores de amostragem, determinando de quais posições espaciais nos quadros de entrada as informações devem ser extraídas para sintetizar o pixel correspondente ao instante intermediário $(t + 0.5)$.

Essa estratégia permite incorporar implicitamente informações de movimento durante o próprio processo de síntese da imagem. Em vez de realizar uma etapa explícita de estimativa de fluxo óptico seguida de reconstrução do quadro, a convolução adaptativa unifica ambas as tarefas em uma única operação. Como resultado, o modelo passa a aprender simultaneamente onde buscar informações e como combiná-las, reduzindo a complexidade do pipeline e possibilitando a geração de quadros intermediários com elevada qualidade visual (NIKLAUS, 2021).

2.2.3 Fluxo Óptico Explícito vs. Kernels de Movimento Implícito

Grande parte dos métodos modernos de Interpolação de Quadros de Vídeo (VFI) baseia-se na estimativa de movimento entre quadros consecutivos. Nesse contexto, o conceito de fluxo óptico desempenha papel fundamental ao descrever o deslocamento aparente dos pixels entre duas imagens ao longo do tempo. Formalmente, o fluxo óptico pode ser representado por um campo vetorial denso, no qual cada pixel possui um vetor (u, v) indicando sua direção e magnitude de deslocamento (SZELISKI, 2022).

Entre os modelos contemporâneos mais relevantes para estimativa de fluxo óptico destaca-se o RAFT (*Recurrent All-Pairs Field Transforms*), proposto por Teed e Deng (2020). O modelo constrói volumes de correlação entre todos os pares de pixels das imagens de entrada e utiliza um mecanismo recorrente para refinar iterativamente os vetores de movimento. Essa estratégia permite atingir elevados níveis de precisão, tornando o RAFT uma das principais referências da área de visão computacional.

Após a estimativa do fluxo óptico, métodos tradicionais de VFI realizam uma etapa adicional denominada *warping*, responsável por deformar ou reposicionar os pixels dos quadros originais de acordo com os vetores calculados. Embora eficaz, essa abordagem pode apresentar dificuldades em regiões de oclusão, desoclusão ou movimentos complexos, nas quais determinados pixels tornam-se visíveis ou invisíveis entre quadros consecutivos (TEED; DENG, 2020; SZELISKI, 2022).

Uma alternativa a esse paradigma consiste na utilização de kernels de movimento implícito, abordagem adotada pelos métodos baseados em convolução adaptativa. Nesse cenário, o movimento não é representado explicitamente por vetores de deslocamento. Em vez disso, a rede neural aprende diretamente um conjunto de coeficientes responsáveis por determinar como os pixels dos quadros de entrada devem ser combinados para sintetizar o valor correspondente ao quadro intermediário (NIKLAUS; MAI; LIU, 2017).

Sob essa perspectiva, os pesos do kernel passam a codificar implicitamente informações de movimento e correspondência espacial. Em vez de calcular uma "seta"indicando o deslocamento de cada pixel, o modelo aprende quais amostras locais devem ser utilizadas e qual contribuição cada uma delas terá na reconstrução do novo quadro. Dessa forma, a estimativa de movimento e a síntese da imagem tornam-se partes de uma única operação computacional.

Essa unificação simplifica o *pipeline* de processamento e elimina a necessidade de etapas explícitas de fluxo óptico e *warping*. Além disso, reduz a propagação de erros entre estágios sucessivos do algoritmo, característica que contribuiu para a popularização das abordagens baseadas em convolução adaptativa na literatura recente de interpolação de quadros de vídeo (NIKLAUS; MAI; LIU, 2017; NIKLAUS, 2021).

2.2.4 Eficiência via Convoluções Separáveis

O principal desafio das abordagens baseadas em convolução adaptativa está relacionado ao tamanho dos kernels necessários para representar movimentos de grande magnitude. Em métodos de interpolação de quadros, cada pixel do quadro intermediário pode depender de uma ampla região dos quadros de entrada, exigindo filtros com dimensões elevadas. Consequentemente, o custo computacional cresce rapidamente à medida que o tamanho do kernel aumenta (NIKLAUS; MAI; LIU, 2017).

Em uma convolução bidimensional convencional, um kernel de dimensão $N \times N$ contém N^2 coeficientes. Isso significa que cada pixel processado requer a combinação de N^2 amostras da imagem de entrada. Para um kernel de 51×51, por exemplo, são necessárias 2.601 operações de combinação para cada pixel sintetizado. Considerando imagens em resolução HD ou Full HD, esse custo torna-se extremamente elevado tanto em processamento quanto em consumo de memória (NIKLAUS; MAI; LIU, 2017; NIKLAUS, 2021).

A técnica de convolução separável busca reduzir essa complexidade explorando uma propriedade matemática específica de determinados filtros. Em vez de representar o kernel como uma matriz bidimensional completa, ele é decomposto em dois filtros unidimensionais independentes: um filtro vertical e um filtro horizontal. Matematicamente, essa decomposição pode ser expressa como:

$$K = v \cdot h^T$$

onde v representa o vetor vertical e h representa o vetor horizontal. Dessa forma, a convolução bidimensional passa a ser executada em duas etapas sucessivas de convolução unidimensional (NIKLAUS; MAI; LIU, 2017).

Como consequência, o número de operações deixa de crescer quadraticamente e passa a crescer linearmente em relação ao tamanho do filtro. Enquanto uma convolução tradicional exige aproximadamente N^2 operações por pixel, a versão separável requer apenas $N + N = 2N$ operações. Para um filtro de dimensão 51×51, isso representa uma redução de 2.601 para apenas 102 operações por pixel.

Essa diferença possui impacto direto na viabilidade computacional do algoritmo. A redução do número de coeficientes diminui a quantidade de acessos à memória, reduz o volume de dados intermediários armazenados durante a inferência e aumenta o potencial de paralelização em arquiteturas modernas de GPU. Como resultado, convoluções separáveis tornam possível utilizar kernels de grande alcance espacial sem que o custo computacional se torne proibitivo (NIKLAUS; MAI; LIU, 2017; KIRK; HWU, 2023).

No contexto deste trabalho, essa propriedade é particularmente relevante, pois a redução da complexidade matemática influencia diretamente a latência de execução e o consumo de memória VRAM durante a inferência. Dessa forma, a separabilidade dos kernels constitui o principal mecanismo que viabiliza a execução de modelos de interpolação de quadros em GPUs de consumo com capacidade limitada de memória.

2.2.5 Implementação de Baixo Nível e I/O de Vídeo

O desenvolvimento de aplicações de processamento de vídeo em alto desempenho exige atenção não apenas aos algoritmos empregados, mas também à forma como os dados são organizados e transferidos entre os diferentes níveis da arquitetura computacional. Nesse contexto, a linguagem C++ é amplamente utilizada em aplicações de visão computacional e computação de alto desempenho por oferecer controle explícito sobre a alocação de memória, estruturas de dados e gerenciamento de recursos do sistema (KIRK; HWU, 2023).

Em aplicações aceleradas por GPU, os dados normalmente são armazenados inicialmente na memória principal do sistema (Host) e posteriormente transferidos para a memória de vídeo da placa gráfica (Device). Essas transferências ocorrem através do barramento PCI Express (PCIe), cuja largura de banda é significativamente inferior à largura de banda interna da memória da GPU. Consequentemente, movimentações excessivas de dados podem se tornar um gargalo relevante para o desempenho da aplicação (NVIDIA Corporation, 2025).

Para minimizar esse problema, é comum organizar os dados em regiões contíguas de memória antes da transferência para o dispositivo gráfico. Essa estratégia melhora a localidade espacial dos acessos, reduz a fragmentação dos dados e favorece operações de leitura e escrita mais eficientes tanto na CPU quanto na GPU (KIRK; HWU, 2023).

No contexto deste trabalho, os quadros de vídeo serão inicialmente carregados utilizando bibliotecas especializadas de entrada e saída de mídia. Após a decodificação, os dados serão reorganizados em estruturas lineares adequadas para processamento paralelo, permitindo transferências eficientes entre memória principal e memória gráfica. Essa etapa é particularmente importante em aplicações de interpolação de quadros, uma vez que múltiplas imagens de alta resolução precisam ser mantidas simultaneamente em memória durante o processo de inferência.

2.2.6 Inferência Desacoplada via ONNX Runtime

O Open Neural Network Exchange (ONNX) consiste em um formato intermediário padronizado para representação de modelos de aprendizado profundo, permitindo interoperabilidade entre diferentes frameworks de treinamento e ambientes de inferência (Linux Foundation AI; Microsoft, 2026). Nesse ecossistema, o ONNX Runtime atua como um mecanismo otimizado de execução, possibilitando a inferência de redes neurais sem dependência direta do framework originalmente utilizado durante o treinamento.

No contexto desta pesquisa, a utilização do ONNX Runtime possui relevância arquitetural, pois permite desacoplar a inferência do ecossistema Python/PyTorch, reduzindo *overheads* relacionados à abstração de alto nível e ao gerenciamento adicional de memória. Além disso, a biblioteca oferece integração nativa com provedores de execução CUDA, permitindo que os tensores permaneçam alocados diretamente na VRAM da GPU durante o pipeline de processamento.

Essa abordagem é particularmente importante para a hipótese investigada neste trabalho, uma vez que a redução de transferências desnecessárias entre memória principal e memória de vídeo constitui um fator relevante para mitigação do gargalo conhecido como *Memory Wall*. Dessa forma, o ONNX Runtime atua não apenas como ferramenta de execução, mas como componente estrutural da estratégia de otimização proposta.

2.2.7 Arquitetura de Decomposição e Garantia de Separabilidade

A viabilidade técnica de utilizar kernels de grandes dimensões (como 51×51) depende da garantia matemática de separabilidade. Para que uma matriz K seja separável, ela deve obrigatoriamente possuir Posto 1 (*Rank-1*), permitindo sua decomposição no produto externo de dois vetores unidimensionais: $K = v \cdot h^T$.

O modelo SepConv resolve esse desafio diretamente em sua arquitetura de rede neural através de uma **bifurcação na camada de saída**. Em vez de estimar os N^2 elementos de uma matriz bidimensional, a rede possui duas ramificações independentes que inferem, simultaneamente, um vetor vertical v $(N \times 1)$ e um vetor horizontal h $(1 \times N)$.

Ao forçar essa estrutura, o algoritmo garante que o kernel resultante seja sempre separável, eliminando erros de aproximação que ocorreriam em uma decomposição analítica

a posteriori. Para a implementação em C++, essa característica é fundamental: o sistema recebe da rede neural apenas $2N$ valores por pixel, permitindo que a operação de convolução seja executada em dois passos lineares sequenciais, reduzindo drasticamente o esforço computacional e a ocupação de memória sem perda de precisão matemática em relação ao modelo bidimensional pretendido.

2.2.8 Arquitetura de GPU e o Gargalo de Memória (*Memory Wall*)

A execução de modelos profundos de visão computacional em hardware de consumo é limitada fundamentalmente pelo tráfego de dados e pela largura de banda de memória. Segundo a teoria arquitetural de Hennessy e Patterson (2022), os sistemas computacionais modernos enfrentam o fenômeno denominado *Memory Wall* (Muro da Memória), caracterizado pela disparidade entre o crescimento da capacidade de processamento aritmético e o aumento da largura de banda dos barramentos de memória (HENNESSY; PATTERSON, 2022). Como consequência, em aplicações intensivas em dados, o tempo despendido na movimentação de tensores entre a memória principal e as unidades lógicas frequentemente supera o tempo de execução das operações matemáticas (HENNESSY; PATTERSON, 2022).

Em Unidades de Processamento Gráfico (GPUs), essa restrição exige estratégias rigorosas de alocação e acesso. Conforme aponta Kirk e Hwu (2023), a vazão máxima de processamento de uma GPU depende de acessos à memória global efetuados de forma contígua e coalescida. Transferências frequentes pelo barramento PCI Express (PCIe) ou realocações dinâmicas de estruturas intermediárias na memória de vídeo (VRAM) introduzem contenção de barramento e paralisam os núcleos de processamento. Esse cenário torna-se crítico em GPUs comerciais limitadas a 8 GB de VRAM, que representam a configuração mais difundida entre usuários de computadores pessoais (Valve Corporation, 2026). Nesses ambientes, a redução do *footprint* espacial da inferência — obtida ao substituir convoluções bidimensionais $O(N^2)$ por filtros separáveis $O(N)$ — constitui um requisito de engenharia essencial para mitigar o *Memory Wall* e evitar o esgotamento do limite físico de memória.

2.2.9 Modelo de Execução Paralela SIMT

Para mitigar os tempos de espera da VRAM e gerenciar milhares de rotinas simultâneas, as arquiteturas de GPU empregam o modelo de execução *Single Instruction, Multiple Threads* (SIMT) (KIRK; HWU, 2023). Diferente do paradigma das CPUs, voltado à redução da latência de instruções sequenciais via memórias *cache* complexas, o modelo SIMT prioriza a vazão total de dados (*throughput*) por meio do paralelismo massivo em nível de dados (KIRK; HWU, 2023; HENNESSY; PATTERSON, 2022).

Sob esse modelo, o hardware organiza as *threads* de execução em grupos lógicos e síncronos denominados *warps* (compostos tipicamente por 32 *threads*) (KIRK; HWU, 2023; NVIDIA Corporation, 2025). O controlador da GPU emite uma única instrução que é executada simultaneamente por todas as *threads* do *warp*, operando sobre diferentes posições de memória ou pixels da imagem (KIRK; HWU, 2023). No contexto do processamento de vídeo e das convoluções adaptativas, esse modelo oferece três vantagens estruturais (GONZALEZ; WOODS, 2009; KIRK; HWU, 2023):

1. **Mapeamento 1:1 de Dados e *Threads*:** A matriz da imagem é particionada diretamente na grade de execução, atribuindo pixels ou vetores unidimensionais de filtros a *threads* específicas, eliminando o *overhead* de laços (*loops*) sequenciais de varredura;

2. **Ocultação de Latência (*Latency Hiding*):** Quando um *warp* é paralisado aguardando uma transação na VRAM, o escalonador do multiprocessador comuta instantaneamente o contexto de execução para outro *warp* cujos dados já estão disponíveis nos registradores, mantendo os núcleos aritméticos continuamente ocupados (KIRK; HWU, 2023);

3. **Coalescência de Memória:** A linearização unidimensional de matrizes de imagem ($index = y \cdot W + x$) permite que *threads* adjacentes de um mesmo *warp* acessem endereços contíguos de VRAM, consolidando múltiplas requisições em uma única transação de barramento (KIRK; HWU, 2023; NVIDIA Corporation, 2025).

2.2.10 Processamento de Baixo Nível e a Plataforma CUDA

A exploração eficiente do modelo SIMT requer interfaces de programação que permitam o controle direto sobre os Multiprocessadores Fluxáveis (*Streaming Multiprocessors* – SMs) da GPU. A plataforma de computação paralela CUDA (*Compute Unified Device Architecture*) fornece extensões à linguagem C++ para a construção de *kernels* — funções executadas paralelamente no dispositivo através de uma hierarquia bidimensional de blocos e *grids* de *threads* (NVIDIA Corporation, 2025).

Desenvolver um motor de inferência nativo em C++ e CUDA traz vantagens determinísticas frente a *frameworks* de alto nível (como PyTorch), que introduzem camadas de abstração e alocações dinâmicas imprevisíveis na memória gráfica:

- **Gerenciamento Determinístico de VRAM:** Permite o controle explícito da alocação de memória e transferência de dados entre o Host (CPU) e o Device (GPU), eliminando reorientações desnecessárias de ponteiros (NVIDIA Corporation, 2025);

- **Exploração da Memória Compartilhada (*Shared Memory*):** Permite que *threads* de um mesmo bloco armazenem temporariamente os coeficientes dos filtros

$1D$ em uma memória *on-chip* de baixíssima latência, reduzindo drasticamente o tráfego na VRAM global (KIRK; HWU, 2023);

- **Aferição Precisa de Desempenho:** Permite isolar o tempo computacional estrito da GPU por meio de eventos da API (`cudaEventRecord`), eliminando o *overhead* do ambiente de execução do Python e a latência de chamadas do sistema operacional (NVIDIA Corporation, 2025).

A integração entre o controle de baixo nível em C++, a complexidade linear $O(N)$ da separabilidade matemática e a execução paralela em CUDA estabelece a infraestrutura necessária para validar experimentalmente os ganhos de latência e a redução do consumo de VRAM em GPUs de consumo.

2.2.11 Métricas de Avaliação Objetiva de Qualidade

A validação de algoritmos de Interpolação de Quadros de Vídeo (VFI) exige a comparação entre o quadro sintetizado (I_{sint}) e o quadro real de referência (I_{real}), também denominado *ground truth*. Para quantificar objetivamente a fidelidade visual das imagens geradas, este trabalho utilizará métricas amplamente adotadas na literatura de processamento de imagens e visão computacional: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM) e Video Multimethod Assessment Fusion (VMAF).

1. Peak Signal-to-Noise Ratio (PSNR):

O PSNR é uma das métricas mais tradicionais para avaliação da qualidade de imagens reconstruídas ou comprimidas. Sua formulação é baseada no Erro Quadrático Médio (*Mean Squared Error* – MSE), que mede a diferença pixel a pixel entre a imagem sintetizada e a imagem de referência (HORE; ZIOU, 2010).

Matematicamente, o PSNR é definido como:

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \tag{2.1}$$

onde MAX_I representa o valor máximo possível de intensidade de um pixel, assumindo normalmente o valor 255 em imagens de 8 bits.

Valores mais elevados de PSNR indicam menor erro de reconstrução e, consequentemente, maior proximidade numérica entre a imagem gerada e a imagem original. Entretanto, embora seja simples e computacionalmente eficiente, essa métrica apresenta limitações perceptuais, pois considera apenas diferenças absolutas entre pixels, sem levar em conta a forma como o sistema visual humano interpreta estruturas, texturas e padrões espaciais (HORE; ZIOU, 2010).

2. Structural Similarity Index (SSIM):

Para contornar as limitações perceptuais do PSNR, Wang et al. (2004) propuseram o *Structural Similarity Index* (SSIM), uma métrica baseada na hipótese de que o sistema

visual humano é mais sensível à preservação de estruturas locais do que a diferenças absolutas de intensidade.

O SSIM compara regiões correspondentes entre duas imagens considerando simultaneamente três componentes fundamentais:

- luminância;
- contraste;
- estrutura.

O resultado é um valor normalmente compreendido no intervalo $[0, 1]$, onde valores próximos de 1 indicam elevada similaridade estrutural entre as imagens comparadas.

Em aplicações de interpolação de quadros, o SSIM é particularmente relevante por ser capaz de identificar degradações visuais frequentemente observadas em quadros sintetizados, como borramento de bordas, perda de detalhes finos e artefatos decorrentes de erros de estimativa de movimento. Por esse motivo, a métrica é amplamente utilizada na literatura contemporânea de VFI como complemento ao PSNR (NIKLAUS; MAI; LIU, 2017; NIKLAUS, 2021).

3. Video Multimethod Assessment Fusion (VMAF):

Desenvolvido pela Netflix, o *Video Multimethod Assessment Fusion* (VMAF) é uma métrica de qualidade perceptual de vídeo baseada em aprendizado de máquina que combina diferentes algoritmos de avaliação em uma única pontuação (LI et al., 2016). Ao contrário de métricas puramente matemáticas como PSNR e SSIM, o VMAF foi projetado especificamente para refletir a percepção humana da qualidade de vídeo em fluxos contínuos.

O modelo funde três indicadores primários de qualidade perceptual:

- **VIF (*Visual Information Fidelity*):** mede a perda de informação de fidelidade visual em múltiplas escalas espaciais;
- **DLM (*Detail Loss Metric*):** mede a perda de detalhes e a presença de artefatos que distraem a atenção do espectador;
- **Motion:** mede o grau de movimento entre quadros consecutivos para ponderar o impacto do borrão temporal.

A saída do VMAF é mapeada em uma escala de 0 a 100, onde valores próximos a 100 indicam qualidade perceptual idêntica à do vídeo de referência. Em algoritmos de VFI, a inclusão do VMAF justifica-se por sua capacidade de avaliar a consistência temporal da sequência de quadros sintetizada, identificando artefatos dinâmicos (como trepidações e descontinuidades) que métricas quadro a quadro nem sempre capturam com precisão.

A utilização conjunta de PSNR, SSIM e VMAF permite avaliar a precisão numérica, a qualidade estrutural e a fidelidade perceptual da sequência gerada. Dessa forma, torna-se possível analisar os impactos das otimizações propostas neste trabalho não apenas sobre a latência e o consumo de memória, mas também sobre a fidelidade visual e a fluidez temporal dos quadros sintetizados.

2.2.12 Mensuração de Desempenho Computacional

Além da qualidade visual dos quadros sintetizados, a avaliação de algoritmos de Interpolação de Quadros de Vídeo (VFI) envolve aspectos relacionados à eficiência computacional. Em aplicações práticas, especialmente em sistemas de tempo real, a qualidade da imagem deve ser analisada em conjunto com o custo necessário para produzi-la, uma vez que algoritmos visualmente precisos podem apresentar inviabilidade operacional devido ao elevado consumo de recursos computacionais (KIRK; HWU, 2023).

Nesse contexto, duas métricas são frequentemente empregadas na literatura para caracterizar o desempenho de sistemas acelerados por GPU: a latência de processamento e o consumo de memória. A latência representa o tempo necessário para que uma determinada operação seja concluída, influenciando diretamente a capacidade do sistema de processar sequências de vídeo em tempo real. Em aplicações de visão computacional, reduções de poucos milissegundos por quadro podem resultar em aumentos significativos na taxa efetiva de processamento (*throughput*) (HENNESSY; PATTERSON, 2022).

Por sua vez, o consumo de memória constitui um fator limitante para a execução de modelos de aprendizado profundo em hardware de consumo. A necessidade de armazenar simultaneamente quadros de entrada, ativações intermediárias e parâmetros do modelo pode levar ao esgotamento da memória disponível, inviabilizando a inferência mesmo quando a capacidade de processamento da GPU permanece ociosa. Por esse motivo, a utilização eficiente da VRAM tornou-se um aspecto relevante no desenvolvimento de aplicações modernas de inteligência artificial (NVIDIA Corporation, 2025).

No contexto da interpolação de quadros baseada em convoluções adaptativas separáveis, a redução da complexidade espacial representa um dos principais fatores que podem contribuir para a diminuição do consumo de memória e para a melhoria da eficiência geral do sistema. Dessa forma, a análise conjunta de métricas de qualidade visual e desempenho computacional permite uma avaliação mais abrangente dos benefícios e limitações da abordagem investigada neste trabalho.

2.3 Trabalhos relacionados

Nesta seção, são apresentados três trabalhos recentes que abordam o desafio da Interpolação de Quadros de Vídeo (VFI). A seleção prioriza métodos que, assim como esta

pesquisa, buscam o equilíbrio entre a fidelidade visual e a eficiência no processamento de grandes deslocamentos de pixels.

2.3.1 Trabalho 1: Revisiting Adaptive Convolutions for Video Frame Interpolation

Niklaus (2021) abordou o problema da complexidade crescente das redes neurais de interpolação, que muitas vezes exigem recursos computacionais excessivos para ganhos marginais de qualidade. Para resolvê-lo, o autor propôs uma revisitação ao método de convoluções adaptativas, introduzindo melhorias sutis na arquitetura de estimativa de kernel. A metodologia baseou-se no refinamento do modelo SepConv original, otimizando a forma como os filtros $1D$ são gerados. Os resultados mostraram que modelos baseados em kernels podem atingir métricas de PSNR e SSIM próximas ao estado da arte com estruturas mais simples. No entanto, a solução ainda depende de frameworks de alto nível (PyTorch) para execução. O presente trabalho diferencia-se por investigar experimentalmente a inferência pura baseada nesses conceitos em C++, focando na eliminação do *overhead* inerente às bibliotecas de aprendizado profundo na alocação de GPU.

2.3.2 Trabalho 2: Motion-Aware Video Frame Interpolation

Han et al. (2024) abordaram o problema da representação inadequada de movimentos complexos em métodos tradicionais de interpolação de quadros, especialmente em cenários com deslocamentos rápidos, mudanças abruptas de direção e regiões de oclusão. Para enfrentar essa limitação, os autores propuseram uma arquitetura baseada em modelagem de movimento multi-escala, capaz de capturar informações espaciais e temporais em diferentes níveis de resolução.

A metodologia combina a extração hierárquica de características com mecanismos de estimativa de movimento que integram informações provenientes de múltiplas escalas da imagem. Essa estratégia permite representar movimentos de grande amplitude sem depender exclusivamente de kernels locais ou de campos de fluxo óptico densos, reduzindo artefatos temporais frequentemente observados em sequências complexas.

Os resultados experimentais demonstraram melhorias na preservação da consistência temporal e na qualidade visual dos quadros sintetizados quando comparados a métodos anteriores de interpolação. Em particular, os autores reportam redução de artefatos em regiões com movimentos rápidos e melhor reconstrução de detalhes estruturais.

Entretanto, a arquitetura proposta prioriza a qualidade visual e a robustez da representação de movimento, sem foco específico na redução do consumo de memória ou na otimização da inferência para hardware de consumo. O presente trabalho diferencia-se ao investigar experimentalmente como a separabilidade de kernels e a inferência desacoplada

via ONNX Runtime podem contribuir para reduzir a latência e o consumo de VRAM em GPUs comerciais, mantendo níveis aceitáveis de fidelidade visual.

2.3.3 Trabalho 3: RAFT: Recurrent All-Pairs Field Transforms for Optical Flow

Teed e Deng (2020) abordou a precisão na estimativa de movimento, um problema seminal para a síntese de novos quadros. Para resolvê-lo, propuseram o modelo RAFT, que utiliza uma arquitetura recorrente para atualizar campos de fluxo óptico através de volumes de correlação de todos os pares de pixels. A metodologia utiliza iterações sucessivas para refinar o movimento detectado. Os resultados estabeleceram um novo estado da arte em termos de precisão de vetores. No entanto, a solução apresenta alta latência e exige GPUs de alto desempenho, limitando severamente a viabilidade em cenários comuns. O presente trabalho diferencia-se por adotar a convolução adaptativa como uma alternativa mais contida ao fluxo óptico denso do RAFT, priorizando a performance em GPU e a otimização do consumo de VRAM através de uma inferência nativa.

Tabela 1 – Comparação entre trabalhos relacionados e a abordagem proposta

Autor	**Artigo**	**Estratégia**	**Limitação / Diferencial**
Teed e Deng (2020)	RAFT: Recurrent All-Pairs Field Transforms for Optical Flow	Fluxo óptico recorrente com atualização iterativa	Elevado custo computacional e alto consumo de VRAM.
Niklaus et al. (2021)	Revisiting Adaptive Convolutions for Video Frame Interpolation	Convoluções adaptativas separáveis com refinamentos na estimativa dos *kernels*	Execução dependente do *framework* PyTorch.
Han et al. (2024)	Motion-Aware Video Frame Interpolation	Modelagem de movimento multi-escala com pirâmide hierárquica e fluxo intermediário	Maior complexidade arquitetural devido ao processamento multiescala.
Este Trabalho	**Inferência SepConv em C++/CUDA**	**Inferência nativa via ONNX Runtime e *kernels* CUDA**	**Análise de latência e uso de VRAM.**

Fonte: Elaborado pelo autor (2026)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, detalhando a arquitetura do motor de inferência, o ambiente experimental, o processo de exportação e desacoplamento do modelo via ONNX Runtime, a implementação do motor em C++ e CUDA, o protocolo experimental, os resultados preliminares obtidos com a primeira versão do motor e o cronograma de trabalho do projeto.

3.1 Solução Proposta e Arquitetura

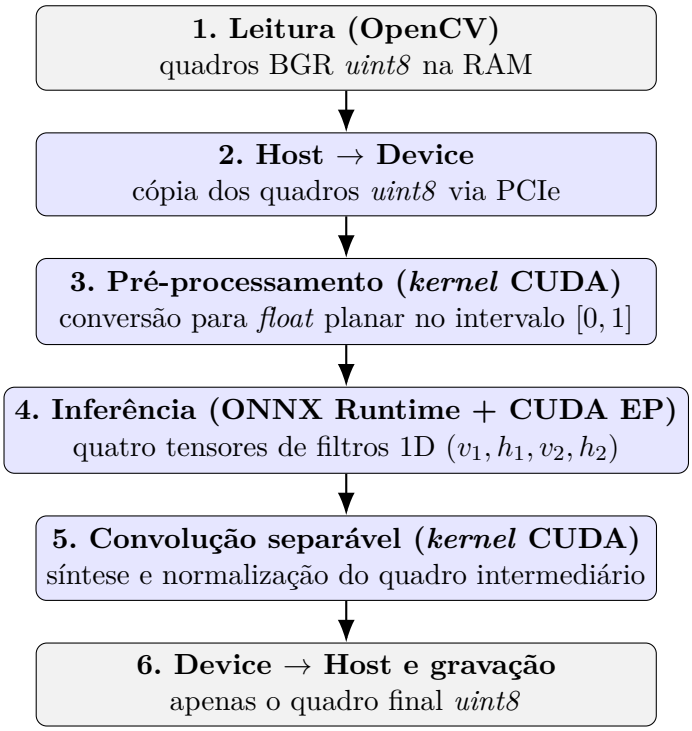
A solução proposta para investigar a viabilidade técnica e a eficiência das convoluções adaptativas separáveis consiste no desenvolvimento de um motor de inferência customizado para Interpolação de Quadros de Vídeo (VFI). O foco é transpor a etapa de inferência do modelo SepConv para um ambiente altamente otimizado em GPU.

O sistema opera em um fluxo de dados (*pipeline*) composto por cinco estágios sequenciais:

1. **I/O (Host):** Leitura e decodificação dos quadros na RAM do sistema via OpenCV (API C++);
2. **Transferência (Host-to-Device):** Envio dos dados linearizados da RAM para a VRAM da GPU via barramento PCIe;
3. **Inferência (ONNX Runtime):** Processamento do modelo pré-treinado SepConv para gerar os vetores separados 1D para cada pixel, mantendo-os alocados nativamente na GPU;
4. **Convolução (CUDA):** Aplicação paralela da convolução separável $O(N)$ utilizando blocos e *threads* CUDA nativamente na VRAM para compor o quadro final;
5. **Retorno e Escrita (Device-to-Host):** Transferência apenas do quadro gerado final de volta para o Host e codificação do vídeo.

A Figura 2 resume o fluxo de dados implementado, destacando quais estágios são executados na memória principal (Host) e quais permanecem inteiramente na memória de vídeo (Device).

Figura 2 – Fluxo de dados do motor de inferência SepConv implementado em C++ e CUDA.



Fonte: Elaborado pelo autor (2026). Em azul, os estágios executados na VRAM.

3.2 Ambiente Experimental

Todos os experimentos foram executados em um computador pessoal com configuração representativa do perfil de hardware de consumo discutido na Seção 1.4. A Tabela 2 descreve o hardware e as versões de software utilizadas, informação necessária para a reprodutibilidade dos resultados.

3.3 Exportação do Modelo e Grafo ONNX

Para desacoplar a inferência do ambiente de treinamento em Python/PyTorch, a arquitetura foi exportada para a plataforma *Open Neural Network Exchange* (ONNX). O modelo em formato `.onnx` recebe dois quadros de entrada e gera como saída os quatro tensores dos filtros unidimensionais (v_1, h_1, v_2, h_2).

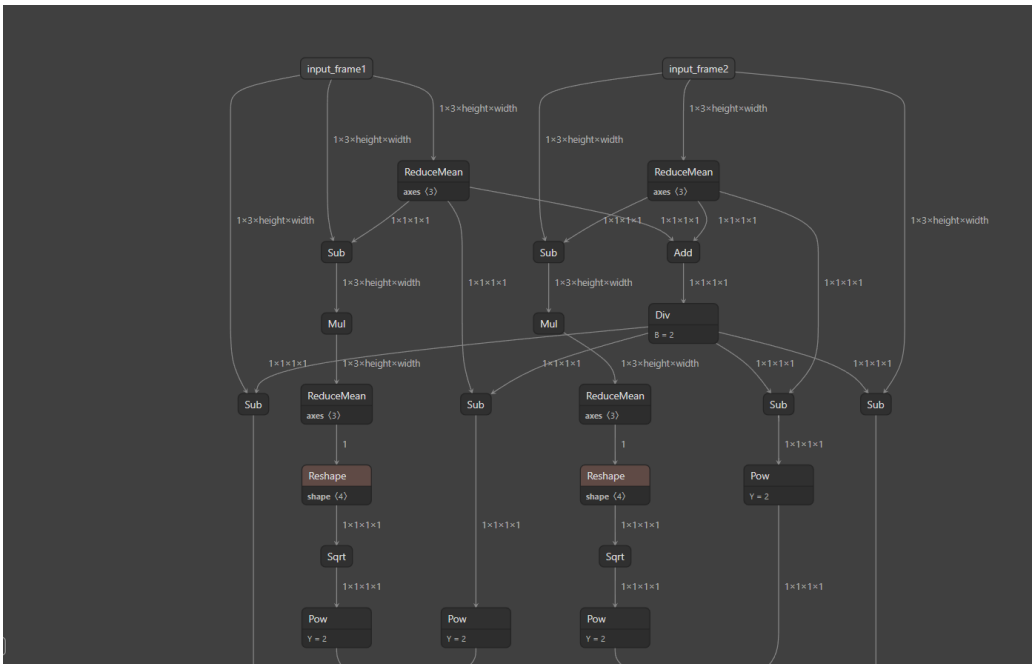
A estrutura do grafo exportado pode ser observada nas Figuras 3 e 4, geradas a partir da inspeção do arquivo executável.

Tabela 2 – Hardware e software do ambiente experimental

Componente	**Especificação**
GPU	NVIDIA GeForce RTX 4060 Ti, 8 GB de VRAM (arquitetura Ada Lovelace, *compute capability* 8.9)
CPU	Intel Core i5-12400F
Memória principal	16 GB
Sistema operacional	Windows 11 Home (*driver* NVIDIA 610.47)
Motor C++	C++17, CUDA Toolkit 13.3, MSVC (Visual Studio 2022), CMake 4.4
Inferência	ONNX Runtime 1.23.2 (GPU) com *CUDA Execution Provider*
Entrada e saída de vídeo	OpenCV 4.13 (API C++ e Python)
Baseline	Python 3.10, PyTorch 2.11 (CUDA 12.8), implementação original de Niklaus (2021)
Métricas	scikit-image 0.25 (PSNR e SSIM), FFmpeg 9.0 com *libvmaf* (VMAF)

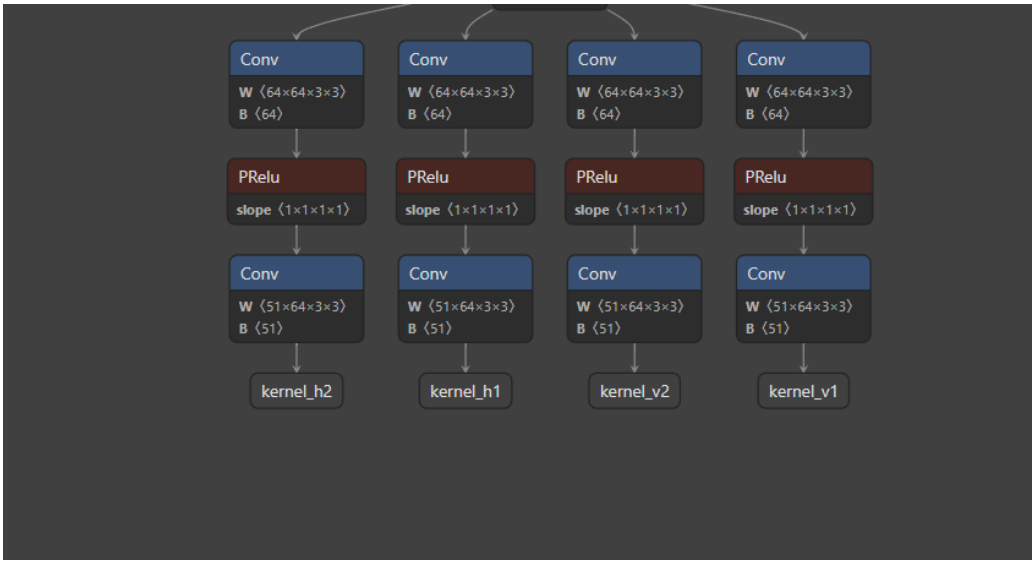
Fonte: Elaborado pelo autor (2026)

Figura 3 – Visualização da camada de entrada do grafo computacional SepConv exportado para ONNX.



Fonte: Elaborado pelo autor (2026)

Figura 4 – Bifurcação das camadas de saída no ONNX emitindo os quatro vetores de filtragem $1D$.



Fonte: Elaborado pelo autor (2026)

3.3.1 Correção do grafo para resoluções arbitrárias

A primeira exportação do modelo funcionava apenas na resolução utilizada durante o processo de exportação (1920 × 1080). Em qualquer outra resolução — como 448 × 256 (Vimeo-90K) ou 1280 × 720 —, a execução era interrompida por incompatibilidade de dimensões em uma operação de soma do decodificador.

A investigação mostrou que o decodificador original recorta a saída de cada etapa de *upsampling* por meio de uma condição em Python que compara as dimensões dos tensores. Durante a exportação, essa condição é avaliada uma única vez e o recorte resultante é fixado no grafo ONNX, que passa a refletir apenas a sequência de dimensões pares e ímpares da resolução de exportação. A correção consistiu em substituir a condição por um fatiamento (*slicing*) até as dimensões do nível correspondente da rede, operação matematicamente equivalente à original e válida para qualquer resolução de entrada com dimensões pares.

Para verificar a equivalência, as saídas do modelo ONNX (executado pelo ONNX Runtime) foram comparadas às da rede original em PyTorch, com entradas idênticas e o modo TF32 desabilitado nas duas implementações. A Tabela 3 mostra que o erro absoluto máximo permaneceu na ordem de 10^{-6} em termos relativos, compatível com a precisão numérica da aritmética de ponto flutuante de 32 bits.

3.4 Implementação do Motor de Inferência

O motor foi implementado como um executável em C++17 que reúne os seis estágios da Figura 2. Todos os *buffers* na VRAM são alocados uma única vez, de acordo com a

Tabela 3 – Diferença entre os filtros 1D estimados pelo modelo ONNX e pela rede original

Resolução	**Erro absoluto máximo**	**Maior valor dos filtros**	**Erro relativo**
448 × 256	$5{,}7 \times 10^{-3}$	$3{,}66 \times 10^{3}$	$1{,}6 \times 10^{-6}$
960 × 540	$2{,}0 \times 10^{-3}$	$3{,}66 \times 10^{3}$	$5{,}3 \times 10^{-7}$
1280 × 720	$8{,}2 \times 10^{-3}$	$4{,}60 \times 10^{3}$	$1{,}8 \times 10^{-6}$
1920 × 1080	$9{,}8 \times 10^{-3}$	$4{,}06 \times 10^{3}$	$2{,}4 \times 10^{-6}$
1920 × 1088	$9{,}8 \times 10^{-3}$	$4{,}02 \times 10^{3}$	$2{,}4 \times 10^{-6}$

Fonte: Elaborado pelo autor (2026)

resolução do vídeo, e reutilizados em todas as interpolações, evitando alocações dinâmicas durante o processamento. O executável opera em dois modos: interpolação de um par de imagens, repetida para fins de medição, e interpolação de vídeos completos com fator 2× ou 4×. No fator 4×, adota-se o mesmo esquema da implementação original: o quadro $t + 0{,}5$ é sintetizado a partir dos quadros originais e os quadros $t + 0{,}25$ e $t + 0{,}75$, a partir dele.

3.4.1 Integração com o ONNX Runtime

O modelo é carregado pela API C++ do ONNX Runtime com o *CUDA Execution Provider*. Para que os tensores não transitem pela memória principal, utiliza-se o mecanismo de *I/O binding*: as duas entradas e as quatro saídas do grafo são associadas diretamente a ponteiros previamente alocados pelo motor na VRAM com `cudaMalloc`. Assim, os quadros pré-processados são lidos pela rede sem cópias adicionais, e os filtros estimados ficam disponíveis na VRAM para o *kernel* de convolução.

3.4.2 *Kernel* de convolução separável

O *kernel* CUDA de síntese atribui uma *thread* a cada pixel (x, y) do quadro intermediário. Sendo I_1 e I_2 os quadros de entrada e v_k, h_k os filtros vertical e horizontal de $N = 51$ coeficientes estimados para o quadro k, cada canal do pixel sintetizado é calculado por:

$$\hat{I}(x,y) = \frac{\displaystyle\sum_{k=1}^{2} \sum_{i=0}^{N-1} v_k(i) \sum_{j=0}^{N-1} h_k(j)\, I_k\Big(c_y(y+i-r),\, c_x(x+j-r)\Big)}{\displaystyle\sum_{k=1}^{2} \Big(\sum_{i} v_k(i)\Big)\Big(\sum_{j} h_k(j)\Big)} \tag{3.1}$$

onde $r = \lfloor N/2 \rfloor = 25$ e c_x, c_y limitam as coordenadas às bordas da imagem. O denominador reproduz a normalização da implementação original, que concatena um canal constante de valor 1 aos quadros e o submete à mesma filtragem; como o *kernel* bidimensional é o produto externo $v\,h^T$, essa soma é obtida diretamente pelo produto das somas dos dois vetores.

Duas decisões de implementação diferem da versão original em PyTorch. Primeiro, a borda replicada de 25 pixels é obtida pela limitação das coordenadas $(c_x,\ c_y)$, sem alocar cópias acolchoadas dos quadros, que no original possuem quatro canais e dimensões $(H + 50) \times (W + 50)$. Segundo, os 51 coeficientes do filtro horizontal são carregados uma única vez em registradores de cada *thread* e reutilizados nas 51 linhas do filtro vertical, e o resultado já é convertido para 8 bits na própria GPU, de modo que apenas o quadro final retorna ao Host.

Cabe uma observação sobre a complexidade discutida na Seção 2.2. Como cada pixel possui o seu próprio par de filtros, não é possível decompor a síntese em duas passadas unidimensionais compartilhadas entre pixels vizinhos, como ocorre em uma convolução separável convencional: o laço da Equação 3.1 ainda realiza N^2 multiplicações por pixel. O ganho da separabilidade, neste contexto, está na quantidade de coeficientes que a rede precisa estimar e manter na memória: $2N = 102$ valores por pixel e por quadro, em vez de $N^2 = 2.601$. Em resolução Full HD, os quatro tensores de filtros em ponto flutuante de 32 bits ocupam cerca de 1,6 GB; filtros bidimensionais completos exigiriam aproximadamente 40 GB, inviáveis em qualquer GPU de consumo.

3.4.3 Instrumentação

O motor mede o tempo de cada estágio com eventos CUDA (`cudaEventRecord`) e, para a inferência — executada pelo ONNX Runtime em seu próprio *stream* e de forma síncrona —, com o relógio monotônico do Host. O consumo de VRAM é obtido pela biblioteca NVML. Como o *driver* do Windows não informa o consumo por processo, registra-se a ocupação total da GPU antes da inicialização do motor (linha de base) e o pico observado após cada inferência; o consumo do motor corresponde à diferença entre ambos. Os resultados de cada execução são acrescentados a um arquivo CSV, a partir do qual são gerados os gráficos e tabelas deste trabalho.

3.4.4 Equivalência com a implementação original

Antes das medições de desempenho, verificou-se se o motor produz o mesmo resultado da implementação original. Para o par de quadros de exemplo distribuído com o código de Niklaus (2021) (584 × 388 pixels), o quadro sintetizado pelo motor C++ apresentou PSNR de 93,67 dB e SSIM de 1,000000 em relação ao quadro gerado pela implementação em PyTorch: apenas 0,003% dos valores diferem, e sempre por um único nível de intensidade, efeito do arredondamento de ponto flutuante. Os dois motores são, portanto, equivalentes do ponto de vista da qualidade visual, e as diferenças medidas a seguir decorrem exclusivamente da forma de execução.

3.5 Validação Experimental e Inspeção de Quadros

Para validar a integridade da síntese temporal e comparar os resultados gerados pelo modelo frente às sequências originais, estruturou-se um ambiente de testes dinâmico baseado em scripts de processamento de imagem.

3.5.1 Sequenciamento de Quadros Interpolados

A validação da interpolação na proporção de 4× (transição de 30 FPS para 120 FPS) é apresentada na Figura 5. Os quadros (a) e (e) correspondem às amostras originais do vídeo (30 FPS), enquanto os quadros (b), (c) e (d) representam as imagens intermediárias sintetizadas pelo motor SepConv.

Figura 5 – Sequência temporal de interpolação 4×: (a) Quadro 840 (t); (b) Quadro 841 ($t + 0.25$); (c) Quadro 842 ($t + 0.50$); (d) Quadro 843 ($t + 0.75$); (e) Quadro 844 ($t + 1$).



(a) Quadro original 840 (t)



(b) Interpolação 841 ($t + 0.25$)



(c) Interpolação 842 ($t + 0.50$)



(d) Interpolação 843 ($t + 0.75$)



(e) Quadro original 844 ($t + 1$)

Fonte: Elaborado pelo autor (2026)

3.5.2 Inspeção de Pixels e Limites de Filtragem

A avaliação de qualidade visual envolveu também a inspeção de regiões críticas de movimento acelerado. A Figura 6 exibe a comparação detalhada entre a região original e o quadro sintetizado, permitindo identificar o nível de fidelidade estrutural e pequenas atenuações de nitidez inerentes às regiões de oclusão.

Figura 6 – Análise comparativa de detalhe: (a) Recorte da região no quadro original 840; (b) Recorte correspondente no quadro interpolado 842.



(a) Original (t) (b) Interpolado ($t + 0.5$)

Fonte: Elaborado pelo autor (2026)

3.6 Protocolo Experimental

A comparação entre o motor proposto e a implementação original segue um protocolo único, aplicado da mesma forma às duas implementações, de modo que as diferenças observadas possam ser atribuídas à forma de execução e não a diferenças de metodologia.

3.6.1 Preparação dos vídeos de teste

Os testes utilizam um trecho do segundo trailer do jogo GTA VI, disponível em resolução 3840 × 2160 a 30 FPS. Como o trailer é composto por muitas cenas curtas, foi realizada uma detecção automática de cortes de cena, que identificou sete transições no intervalo de 20 segundos disponível. Cortes de cena não possuem movimento a ser interpolado e reduzem artificialmente as métricas de qualidade — em uma execução preliminar, um único corte produziu um quadro com PSNR de 12,5 dB. Por isso, foi selecionado um trecho de 61 quadros (aproximadamente 2 segundos) contido no maior intervalo sem cortes.

O trecho foi convertido para quatro resoluções: 1280 × 720 (720p), 1920 × 1080 (1080p), 2560 × 1440 (1440p) e 3840 × 2160 (2160p). Todos os vídeos de teste e de saída são gravados com o codec sem perdas FFV1, para que a compressão não interfira nas métricas de qualidade.

3.6.2 Procedimento de medição

Para cada resolução, gera-se uma versão do trecho contendo apenas os quadros pares (15 FPS). Cada implementação interpola essa versão com fator 2×, reconstruindo um vídeo de 30 FPS no qual os quadros de índice ímpar são sintetizados. Esse procedimento permite medir o desempenho e a qualidade em uma única execução: os quadros sintetizados são comparados aos quadros reais correspondentes do trecho original, que funcionam como *ground truth*.

As duas implementações leem e gravam os vídeos com a mesma biblioteca (OpenCV) e o mesmo esquema de interpolação. As três primeiras interpolações de cada execução são descartadas como aquecimento — período em que ocorrem a inicialização do contexto CUDA, a seleção de algoritmos do cuDNN e a reserva inicial de memória —, e as demais (27 por resolução) compõem a média e o desvio padrão do tempo por quadro. Em 2160p, devido ao tempo de execução, foram processados apenas cinco quadros, com uma interpolação de aquecimento.

3.6.3 Métricas

- **Latência**: tempo médio para sintetizar um quadro, incluindo a transferência dos quadros para a GPU, a inferência, a convolução e o retorno do resultado ao Host;
- **Consumo de VRAM**: pico de ocupação da GPU medido pela NVML menos a ocupação anterior ao início da execução. No Windows, quando a VRAM se esgota, o *driver* passa a utilizar a memória principal do sistema (*Sysmem Fallback*) em vez de interromper a execução, o que eleva drasticamente a latência. Como a NVML não contabiliza essa memória, as execuções em que a VRAM foi saturada são sinalizadas: no motor C++, quando a ocupação atinge 97% da capacidade; na *baseline*, também quando a memória reservada pelo PyTorch excede a VRAM livre no início da execução;
- **Fidelidade visual**: PSNR, SSIM e VMAF calculados exclusivamente sobre os quadros sintetizados. Incluir os quadros pares, que são cópias dos originais, elevaria artificialmente os resultados. Pelo mesmo motivo, as tarjas pretas horizontais presentes no trailer (*letterbox*, 20% da altura do quadro), reconstruídas sem erro por qualquer método, são excluídas da área avaliada. O VMAF é calculado pelo filtro *libvmaf* do FFmpeg com o modelo padrão `vmaf_v0.6.1`; como verificação, a comparação do vídeo de referência com ele mesmo resultou em 99,91, valor esperado para sequências idênticas.

3.7 Resultados Preliminares

Esta seção apresenta os resultados da primeira versão do motor C++/CUDA, que reproduz fielmente a estratégia de memória da implementação original: todos os tensores do quadro inteiro são processados de uma só vez. A Tabela 4 consolida as medições, detalhadas nas subseções seguintes.

Tabela 4 – Desempenho e fidelidade visual por resolução (interpolação 2×)

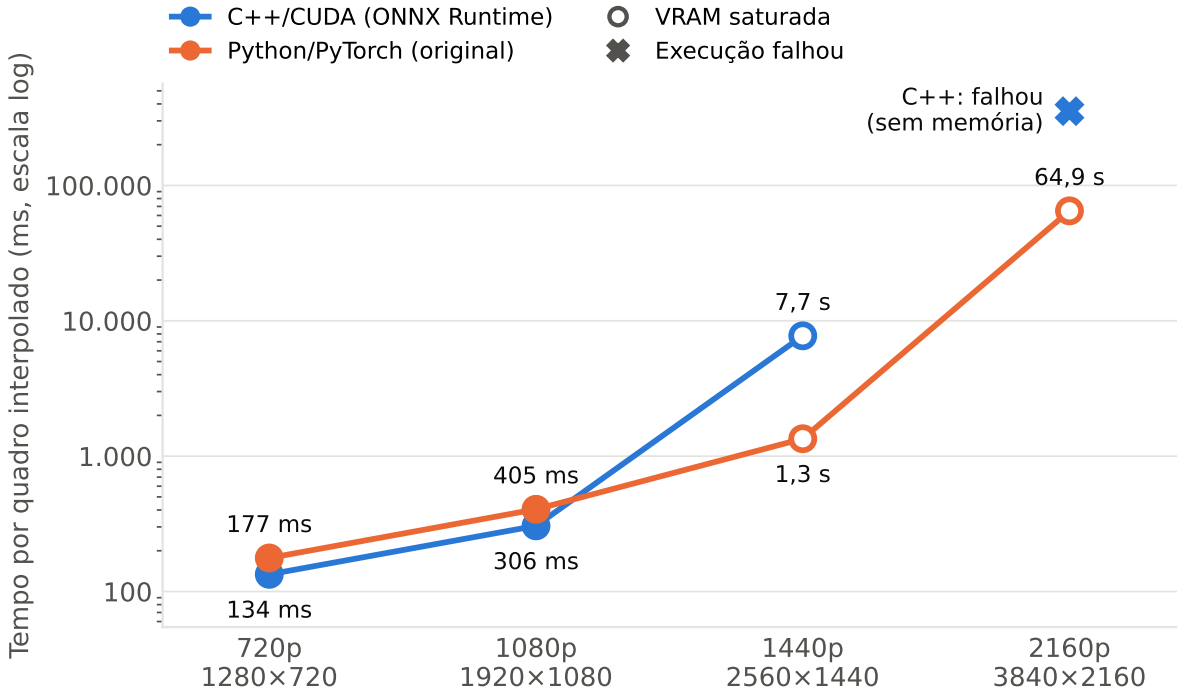
Resolução	**Implementação**	**ms/quadro**	**VRAM (MiB)**	**PSNR (dB)**	**SSIM**	**VMAF**
720p	C++/CUDA	133,9	3.138	34,48	0,9622	85,16
720p	Python/PyTorch	176,8	2.149	34,48	0,9622	85,16
1080p	C++/CUDA	305,5	6.648	34,10	0,9538	77,42
1080p	Python/PyTorch	404,9	4.585	34,10	0,9538	77,42
1440p	C++/CUDA	7.745,1*	7.264*	34,04	0,9532	71,91
1440p	Python/PyTorch	1.344,5*	7.324*	34,04	0,9532	71,91
2160p	C++/CUDA	falhou por falta de memória				
2160p	Python/PyTorch	64.900,5*	7.133*	31,44	0,9323	50,33

* VRAM saturada: o excedente foi alocado na memória principal e não é contabilizado pela NVML. Fonte: Elaborado pelo autor (2026)

3.7.1 Latência

A Figura 7 apresenta o tempo médio por quadro interpolado. Em 720p e 1080p, resoluções em que a VRAM foi suficiente para as duas implementações, o motor C++ foi cerca de 24% mais rápido que a *baseline* (133,9 ms contra 176,8 ms e 305,5 ms contra 404,9 ms), com desvio padrão inferior a 1% da média. A partir de 1440p, a VRAM se esgota nas duas implementações e o comportamento se inverte: o motor C++ passa a levar 7,7 s por quadro, cerca de 5,8 vezes mais que a *baseline* (1,3 s), e em 2160p não consegue alocar memória para a inferência. A implementação em PyTorch conclui o 2160p, porém com 64,9 s por quadro, pois a maior parte dos tensores passa a residir na memória principal.

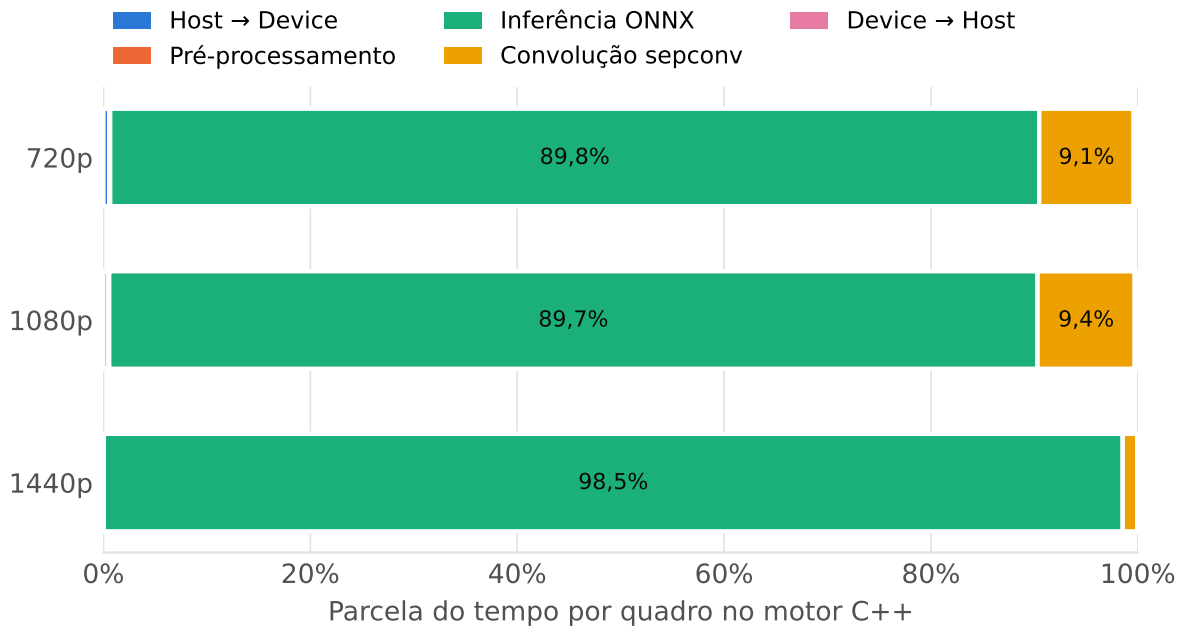
Figura 7 – Tempo médio por quadro interpolado em função da resolução (escala logarítmica).



Fonte: Elaborado pelo autor (2026). Marcadores vazados indicam VRAM saturada.

A Figura 8 mostra como o tempo do motor C++ se distribui entre os estágios do *pipeline*. A inferência da rede pelo ONNX Runtime responde por cerca de 90% do tempo (274,2 ms em 1080p), seguida pela convolução separável (28,7 ms, 9,4%); as transferências entre Host e Device e o pré-processamento somam menos de 1%. Esse resultado indica que, nesta versão, a transferência pelo barramento PCIe não é um gargalo relevante, e que os ganhos de latência dependem principalmente da etapa de inferência. Em 1440p, a participação da inferência sobe para 98,5%, consequência direta do uso da memória compartilhada.

Figura 8 – Distribuição do tempo por quadro entre os estágios do motor C++.

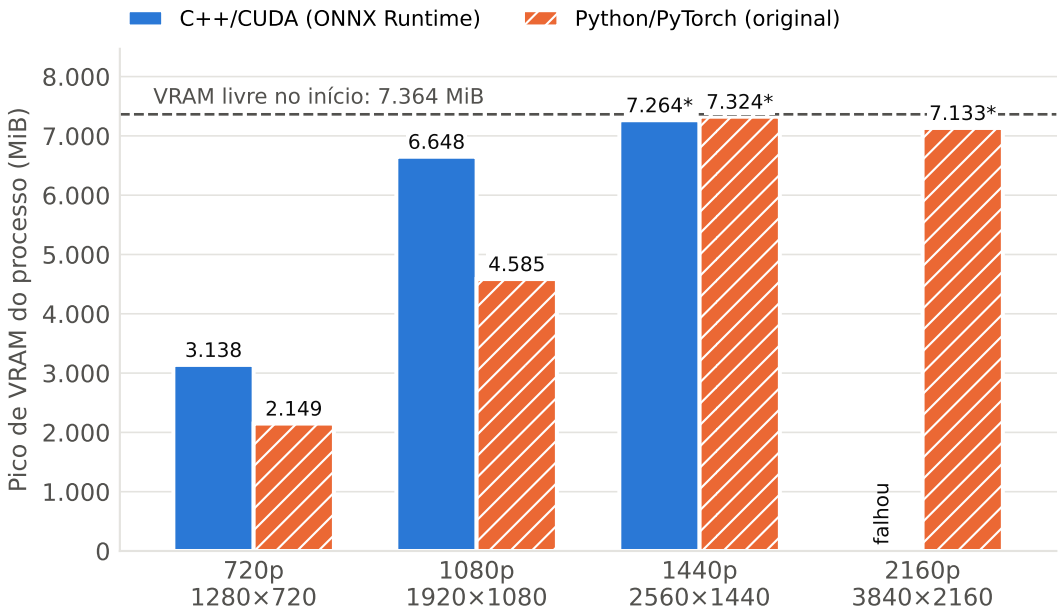


Fonte: Elaborado pelo autor (2026)

3.7.2 Consumo de VRAM

Ao contrário do esperado pela hipótese deste trabalho, a primeira versão do motor C++ consumiu **mais** VRAM que a *baseline* (Figura 9): 3.138 MiB contra 2.149 MiB em 720p e 6.648 MiB contra 4.585 MiB em 1080p, aproximadamente 45% a mais. Duas causas prováveis, a serem isoladas nas próximas etapas, foram identificadas na análise da implementação. Primeiro, o motor mantém alocados durante toda a execução os quatro tensores de filtros (cerca de 1,6 GB em 1080p), enquanto o PyTorch libera os tensores intermediários assim que deixam de ser utilizados. Segundo, o ONNX Runtime reserva memória para os tensores intermediários da rede em um alocador próprio (*arena*) que não a devolve entre execuções. Ajustes das opções do *CUDA Execution Provider* relacionadas ao espaço de trabalho do cuDNN não alteraram o consumo de forma perceptível.

Figura 9 – Pico de VRAM do processo em função da resolução.



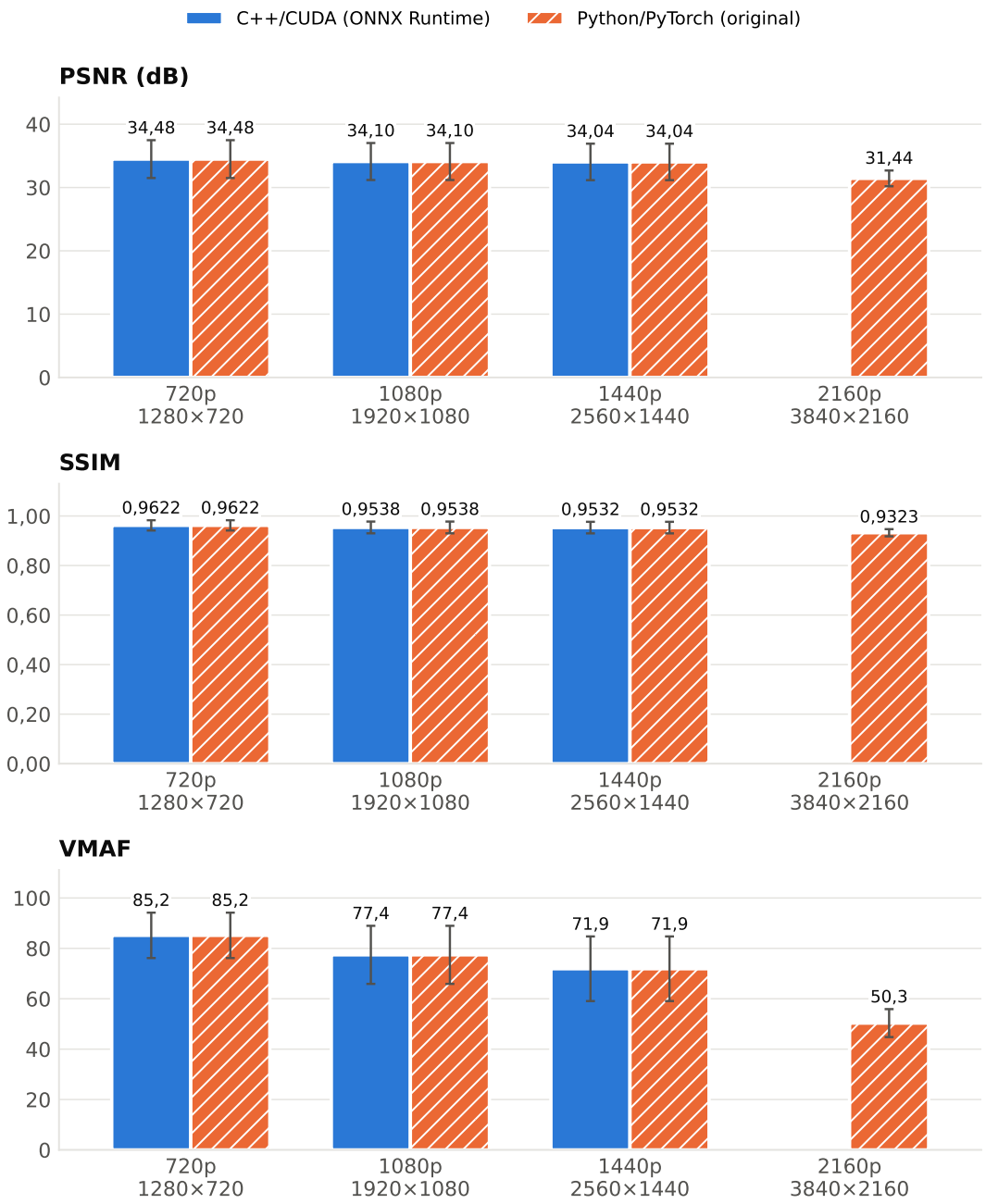
* VRAM saturada: o excedente vai para a memória compartilhada do sistema e não é medido pela NVML; o consumo real é maior que o valor mostrado.

Fonte: Elaborado pelo autor (2026)

3.7.3 Fidelidade visual

As duas implementações produziram resultados idênticos em todas as resoluções avaliadas: PSNR e SSIM coincidem até a quarta casa decimal, e o VMAF difere em menos de 0,001 ponto (Figura 10). Esse resultado é coerente com a equivalência verificada na Seção 3.4 e confirma que a mudança da forma de execução não degradou a qualidade dos quadros sintetizados.

Figura 10 – Fidelidade visual dos quadros sintetizados: PSNR, SSIM e VMAF médios por resolução, com o desvio padrão entre quadros.



Fonte: Elaborado pelo autor (2026)

Observa-se, por outro lado, uma queda da qualidade com o aumento da resolução. Como em 2160p foram avaliados apenas os quatro primeiros quadros sintetizados, a comparação justa entre resoluções deve considerar os mesmos quadros: nesse recorte, o PSNR cai de 32,48 dB (720p) para 31,44 dB (2160p), uma redução de cerca de 1 dB, enquanto o VMAF cai de 78,88 para 50,33. Duas hipóteses, ainda não verificadas, podem explicar o comportamento. A primeira é o alcance limitado dos filtros: com $N = 51$, o modelo representa deslocamentos de até 25 pixels em cada direção, e o mesmo movimento da cena

ocupa, em 2160p, três vezes mais pixels que em 720p. A segunda é metodológica e afeta especificamente o VMAF: o modelo utilizado foi calibrado para a visualização de conteúdo em 1080p, e há um modelo específico para 4K.

A Figura 11 apresenta um exemplo qualitativo em 1080p. Para evitar a escolha de um caso favorável, foi utilizado o quadro sintetizado de PSNR mediano. O mapa de erro mostra que as diferenças em relação ao quadro real se concentram nas bordas de objetos em movimento — como o contorno do personagem e de seu braço — e em texturas finas, enquanto as regiões estáticas são reconstruídas com erro próximo de zero.

Figura 11 – Exemplo de quadro sintetizado em 1080p (quadro de PSNR mediano): (a) quadro real; (b) quadro sintetizado pelo motor C++; (c) mapa do erro absoluto.

(a) Quadro real 47 (ground truth)



(b) Quadro sintetizado 47 — PSNR 33,09 dB, SSIM 0,9510

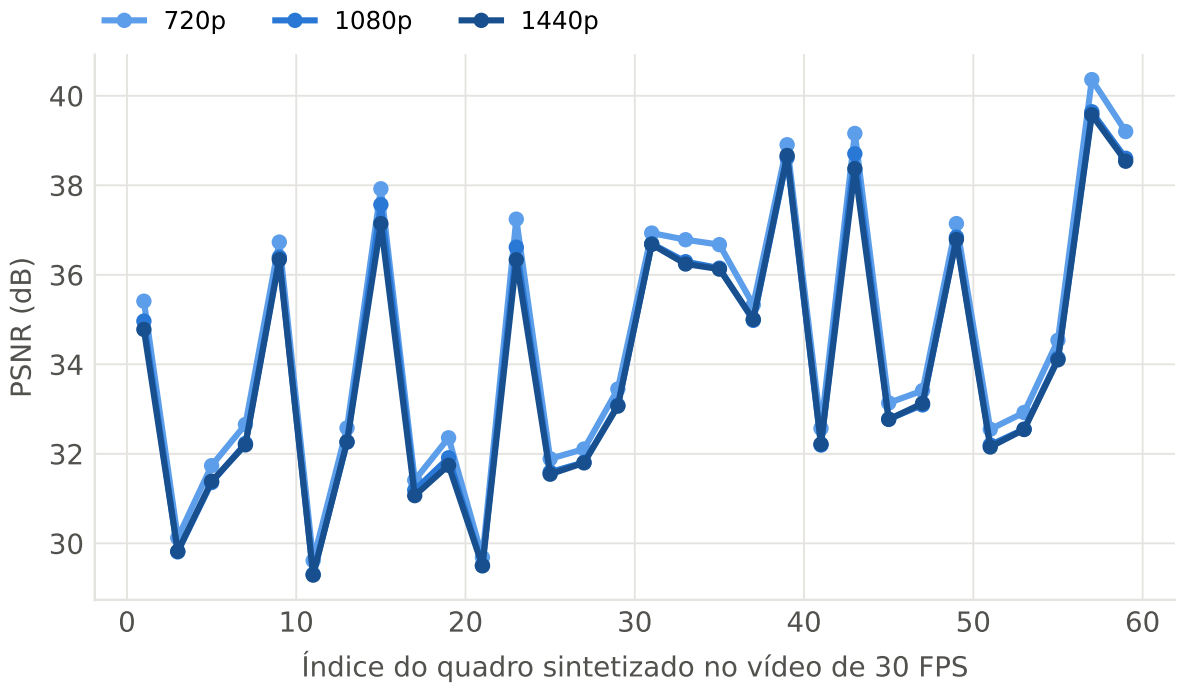


(c) Erro absoluto |a − b| (média dos canais)



A Figura 12 detalha o PSNR de cada quadro sintetizado. As curvas das três resoluções têm o mesmo formato, o que indica que a dificuldade da interpolação é determinada principalmente pelo conteúdo de cada instante (intensidade do movimento), e não pela resolução.

Figura 12 – PSNR de cada quadro sintetizado pelo motor C++, por resolução.



Fonte: Elaborado pelo autor (2026)

3.7.4 Discussão e próximos passos

Os resultados preliminares confirmam parcialmente a hipótese formulada na Seção 1.3. A redução de latência foi observada nas resoluções em que a VRAM é suficiente, sem qualquer perda de fidelidade visual. A redução do consumo de VRAM, porém, não se verificou: a simples transposição da inferência para C++ e ONNX Runtime, mantendo o processamento do quadro inteiro, consome mais memória que a implementação original e degrada mais rapidamente quando a memória se esgota. Esse resultado indica que o benefício esperado não decorre automaticamente da troca de linguagem ou de *framework*, mas depende de um controle explícito do uso de memória.

Com base nesse diagnóstico, as próximas etapas do desenvolvimento são:

1. **Processamento em blocos (*tiling*)**: dividir o quadro em regiões com sobreposição suficiente para o alcance dos filtros e para o campo receptivo da rede, limitando o pico de VRAM independentemente da resolução. É a estratégia com maior potencial para viabilizar 1440p e 2160p dentro do limite de 8 GB;

2. **Precisão reduzida (FP16)**: avaliar a redução de memória e de latência obtida com a inferência em meia precisão, medindo o impacto correspondente nas métricas de fidelidade;

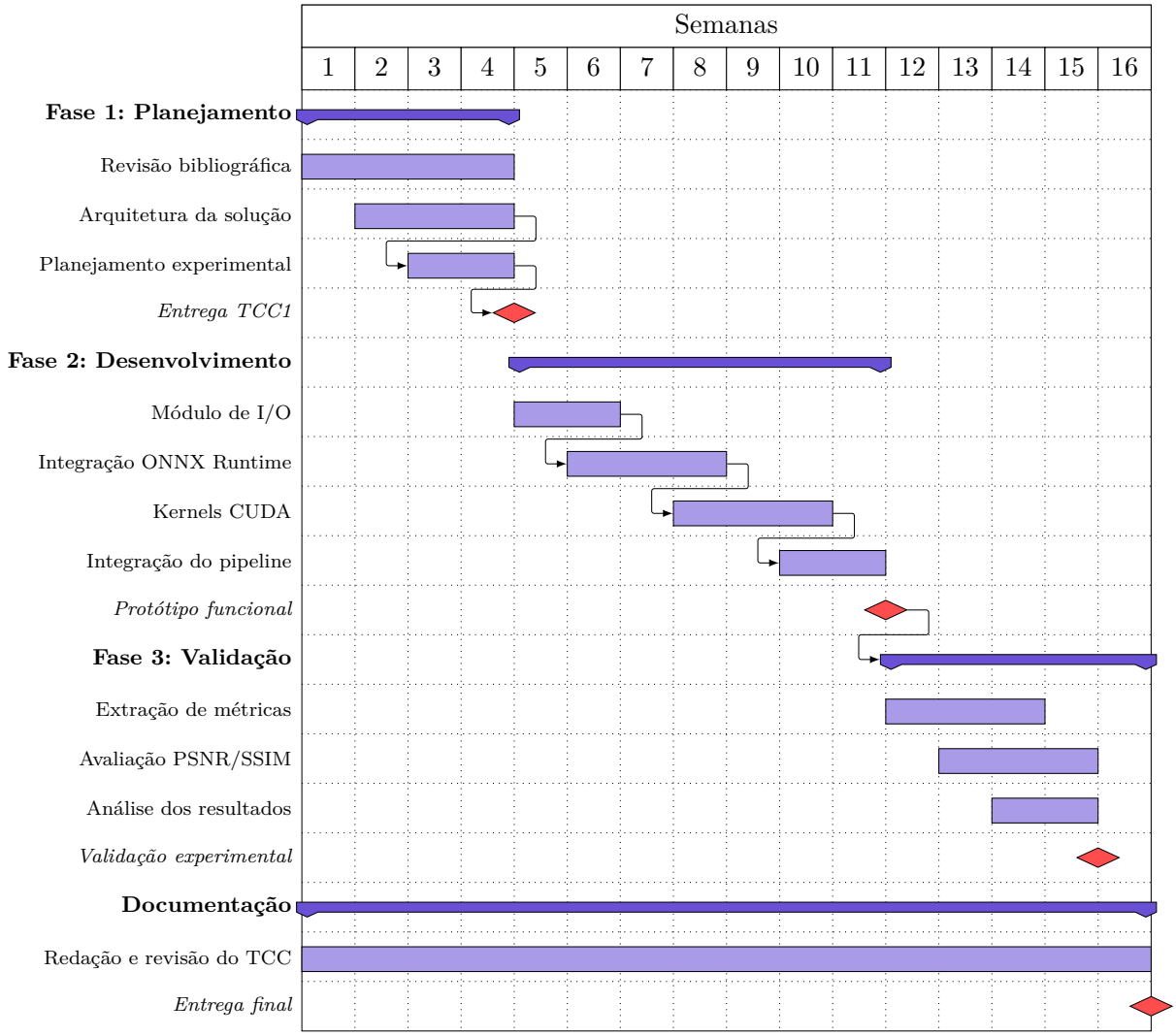
3. **Configuração do alocador do ONNX Runtime**: avaliar estratégias de reserva de memória e a liberação da *arena* entre execuções;

4. **Ampliação da base de avaliação**: repetir as medições em sequências do Vimeo-90K (XUE et al., 2019) e em mais trechos de vídeo, com mais quadros em 2160p e com o computador dedicado exclusivamente aos testes, para fortalecer a validade estatística dos resultados.

3.8 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Figura 13 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

Referências

BAO, W. et al. Depth-aware video frame interpolation. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2019. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/1904.00830>. 8

BORDWELL, D.; THOMPSON, K.; SMITH, J. *Film Art: An Introduction*. 11. ed. [S.l.]: McGraw-Hill Education, 2016. 7

GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3. ed. São Paulo: Pearson, 2009. 8, 14, 15, 20

HAN, P. et al. Motion-aware video frame interpolation. *Neural Networks*, v. 178, p. 106433, 2024. ISSN 0893-6080. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0893608024003575>. 25

HENNESSY, J. L.; PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 7. ed. [S.l.]: Morgan Kaufmann, 2022. 20, 24

HORE, A. C.; ZIOU, D. Image quality metrics: Psnr vs. ssim. *20th International Conference on Pattern Recognition*, p. 2366–2369, 2010. 22

KIRK, D. B.; HWU, W. mei W. *Programming Massively Parallel Processors: A Hands-on Approach*. 5. ed. [S.l.]: Morgan Kaufmann, 2023. 15, 18, 20, 21, 24

LI, Z. et al. *Toward A Practical Perceptual Video Quality Metric*. [S.l.], 2016. Acesso em: 25 ago. 2026. Disponível em: <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>. 23

Linux Foundation AI; Microsoft. *ONNX Runtime Documentation*. 2026. Acesso em: 28 maio 2026. Disponível em: <https://onnxruntime.ai/>. 19

NIKLAUS, S. Revisiting adaptive convolutions for video frame interpolation. In: *IEEE Winter Conference on Applications of Computer Vision (WACV)*. [s.n.], 2021. Acesso em: 12 abr. 2026. Disponível em: <https://openaccess.thecvf.com/content/WACV2021/html/Niklaus_Revisiting_Adaptive_Convolutions_for_Video_Frame_Interpolation_WACV_2021_paper.html>. 16, 17, 23, 24, 29, 32

NIKLAUS, S.; MAI, L.; LIU, F. Video frame interpolation via adaptive separable convolution. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2017. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/1708.01692>. 7, 8, 16, 17, 18, 23

NVIDIA Corporation. *CUDA C++ Programming Guide*. 2025. Acesso em: 15 maio 2026. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. 18, 20, 21, 24

SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2. ed. New York: Springer, 2022. 7, 15, 16

TEED, Z.; DENG, J. Raft: Recurrent all-pairs field transforms for optical flow. In: *European Conference on Computer Vision (ECCV)*. [s.n.], 2020. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/2003.12039>. 7, 16, 25

Valve Corporation. *Steam Hardware & Software Survey*. 2026. Acesso em: 28 maio 2026. Disponível em: <https://store.steampowered.com/hwsurvey>. 20

WANG, Z. et al. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, IEEE, v. 13, n. 4, p. 600–612, 2004. 22

WATKINSON, J. *The Art of Digital Video*. 4. ed. Oxford: Focal Press, 2009. 7

XUE, T. et al. Video enhancement with task-oriented flow. In: *International Journal of Computer Vision*. [S.l.: s.n.], 2019. v. 127, n. 8, p. 1106–1125. 45